

Universidad San Jorge

Escuela de Arquitectura y Tecnología

**Grado en Diseño y Desarrollo de
Videojuegos**

Proyecto Final

**Generación procedural de mapas en un
entorno 2D**

Autor del proyecto: Héctor Anguita

Director del proyecto: Sergio Jimeno

Zaragoza, 9 de Septiembre de 2025



Este trabajo constituye parte de mi candidatura para la obtención del título de Graduado en Diseño y Desarrollo de Videojuegos por la Universidad San Jorge y no ha sido entregado previamente (o simultáneamente) para la obtención de cualquier otro título.

Este documento es el resultado de mi propio trabajo, excepto donde de otra manera esté indicado y referido.

Doy mi consentimiento para que se archive este trabajo en la biblioteca universitaria de Universidad San Jorge, donde se puede facilitar su consulta.

Firma

Fecha

09/09/2025

Dedicatoria y Agradecimiento

Quisiera expresar mi más sincero agradecimiento a todas las personas que han hecho posible la realización de este trabajo.

En primer lugar a mi director de proyecto, Sergio Jimeno, por su ayuda a lo largo del mismo, y guiarme para darle forma al trabajo realizado. Su experiencia en el tema ha sido de gran ayuda para la realización del mismo.

A mis profesores y profesoras de la Universidad San Jorge, por haberme transmitido a lo largo de estos años conocimientos y la pasión por el diseño y el desarrollo de los videojuegos.

A mis amigos que me han tenido que escuchar largo y tendido sobre mi trabajo, aunque muchos de ellos no lo entendiesen.

A mi familia por todo el apoyo a lo largo de toda mi vida de estudios, su paciencia y su ánimo.

Y, como no, a los desarrolladores de los juegos en los cuales me he inspirado e investigado acerca de sus juegos o su trabajo.

Sin vosotros, esto no habría sido posible.

Muchas gracias a todos.



Tabla de contenido

Resumen.....	1
Abstract.....	2
1. Introducción.....	3
2. Antecedentes, Estado del Arte y Contexto.....	4
2.1. Información General.....	4
2.2. Inicios.....	5
2.3. Actualidad.....	9
2.4. Algoritmos.....	15
3. Objetivos.....	27
4. Metodología.....	28
5. Implementación.....	29
5.1. Implementación Algoritmo de Grafos.....	29
5.2. Implementación Algoritmo de Partición Binaria del Espacio (BSP).....	36
6. Estudio Económico.....	42
7. Resultados y evaluación.....	45
8. Conclusiones.....	49
9. Bibliografía.....	50
10. Anexos.....	53
10.1. Propuesta del TFG.....	53
10.2. Acta de reuniones.....	54
10.3. Cambios respecto a la entrega intermedia.....	60

Resumen

La generación procedural de contenido (PCG) es una herramienta muy útil en la creación de videojuegos, ya que permite crear entornos dinámicos, únicos y variados. En este trabajo se investiga y se implementa en Unity dos técnicas para la generación de mapas en 2D, donde se usa el algoritmo de grafos, con diferentes métodos de generación como Prim y Kruskal, y el algoritmo de partición binaria del espacio (BSP).

A su vez, se pasa por el contexto histórico y el estado del arte de la PCG, y también se hace un análisis comparativo entre los dos algoritmos mencionados anteriormente, evaluando su rendimiento.

Gracias a esto, se proporciona una base y una guía que pueda servir en un futuro con la investigación de la generación procedural a las personas que estén interesadas en el tema, pero que no sepan cómo empezar a desarrollar su trabajo. O que quieran aprender acerca de la generación procedural, pero no sepan muy bien cómo entrar a este fantástico mundo.

Palabras clave: Generación procedural de contenido (PCG), rendimiento, algoritmos, Prim, Kruskal, grafos, BSP

Abstract

Procedural Content Generation (PCG) is a very useful tool in video game development, as it allows for the creation of dynamic, unique, and varied environments. This work investigates and implements in Unity two techniques for 2D map generation, a graph algorithm, using different generation methods such as Prim and Kruskal, and the Binary Space Partitioning (BSP) algorithm.

Furthermore, the historical context and state of the art of PCG are covered, and a comparative analysis is carried out between the two aforementioned algorithms, evaluating their performance.

Thanks to this work, I bring a guide that may help in the future for research on procedural generation to those that are interested in this topic, but unsure on how to begin developing the work. Or just to those that want to learn about procedural generation, but are not sure how to enter to this fantastic world

Keywords: Procedural Content Generation (PCG), performance, algorithms, Prim, Kruskal, graphs, BSP

1. Introducción

La generación procedural ha marcado un antes y un después en el desarrollo de videojuegos, permitiendo así la creación dinámica de mundos sin la necesidad de tener que crear y diseñar todo desde cero, y posibilitando también así tener mayor material con el que trabajar.

Particularmente, hablando de la generación procedural de mapas 2D, ha sido una técnica que se ha usado ampliamente en géneros como los roguelikes, o juegos de estrategia y exploración donde es importante la aleatoriedad para que el usuario pueda tener un mayor disfrute a la hora de poder rejugar el título.

Este TFG surge de la necesidad de evaluar y comparar diferentes métodos de generación procedural para mapas en 2D, viendo que algoritmo podría ser mejor y en qué circunstancias, para que los resultados puedan servir como guía en un futuro para más investigaciones o ayuda para desarrolladores que busquen integrar estas técnicas en sus proyectos.

Para la implementación del código y la creación de los mapas, se ha utilizado el motor gráfico Unity, en específico la versión 6.000.0.18f1 y Visual Studio 2022.

2. Antecedentes, Estado del Arte y Contexto

2.1. Información General

La generación procedural es una técnica muy usada en el diseño moderno de videojuegos, permitiendo crear experiencias dinámicas y únicas sin depender de contenido manualmente diseñado. Su importancia radica no solo en la eficiencia de desarrollo, reduciendo costes y tiempo de producción, sino también en su capacidad para enriquecer la rejugabilidad, como demuestran títulos clásicos como Rogue o contemporáneos como Dead Cells y Terraria.



Fig. 1 Fragmento del juego Dead Cells. Fuente:
<https://www.rockpapershotgun.com/four-years-on-dead-cells-remains-a-roguelike-delight>

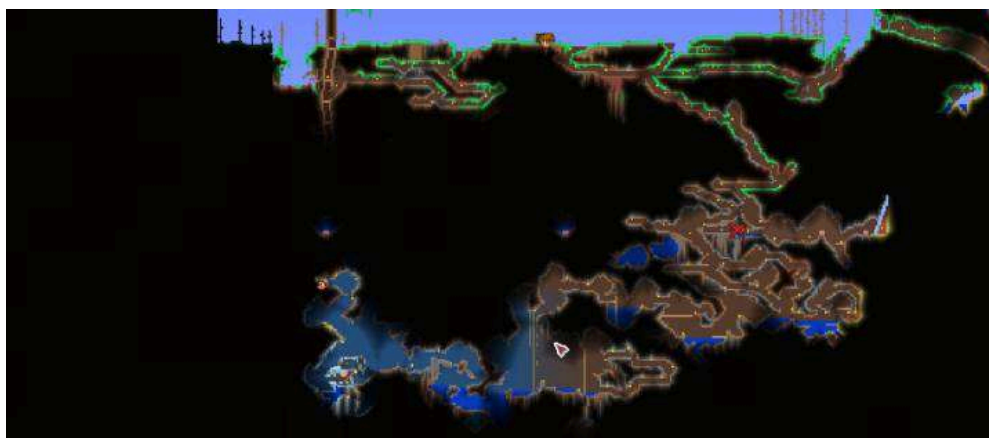


Fig. 2 Mapa de las cuevas que se pueden generar en Terraria. Fuente:
<https://www.carlsguides.com/terraria/walkthrough/exploring-underground-caverns.php>

Pero no todo es bueno ni se puede hacer con cada aspecto del juego, ya que se puede correr el riesgo de perder completamente el control del mismo si se dejan muchos elementos al azar. Al dar demasiada libertad, se puede producir una ruptura o superposición, o valores tan alejados uno del otro que sean incompatibles o imposibles de materializar.

A medida que los videojuegos evolucionan, su nivel de complejidad aumenta, y con ello el coste monetario para crear los videojuegos de forma manual, sobre todo en juegos AAA en los que se invierten decenas de millones para crearlos.

Es por ello que la generación procedural es muy útil y cada vez más popular para abaratar costes y tiempo.[1]

2.2. Inicios

En un principio, la generación procedural, no se planteó para que el usuario tuviese experiencias nuevas e ilimitadas cada vez que entrase a jugar, sino que los primeros videojuegos gráficos fueron severamente frenados por limitaciones de memoria.

Esto provocó que el contenido, como mapas, fuera generado algorítmicamente sobre la marcha, ya que, simplemente, no había espacio suficiente para almacenar una gran cantidad de niveles preconfeccionados y el arte de los niveles. Utilizaban generadores con números pseudoaleatorios con valores predefinidos de semillas aleatorias con el fin de crear grandes mundos de juego que parecieran prefabricados.

El primer juego de rol, no es el propio juego Rogue, que salió en 1980, si no Beneath Apple Manor que salió 2 años antes, el creador de Rogue no sabía de la existencia de este juego previo al suyo y por eso le puso el nombre que finalmente se siguió usando para los juegos de rol a lo largo de los años, también conocidos como roguelike o roguelite a día de hoy.

Este juego creó mazmorras con caracteres ASCII en su inicio, aunque en 1982 crearon una versión con mejores gráficos.

Genera proceduralmente las habitaciones, pasillos, monstruos y el tesoro final que el jugador tenía que obtener.[2][3]



Fig. 3 Ejemplo del mapa inicial con caracteres ASCII a la izquierda y a la derecha la mejora gráfica. Fuentes:
<https://crpgbook.wordpress.com/review-index/1978-beneath-apple-manor/>
<https://dosgames.com/game/beneath-apple-manor/>

El juego Rogue también generaba proceduralmente las mazmorras, los monstruos y los tesoros, por lo que, es muy parecido al juego anterior, incluso en la estética del juego, ya que, al crearse en años similares, aunque sin saber la existencia del otro, tampoco se podían hacer juegos muy diferentes en cuestión de estética.[4]

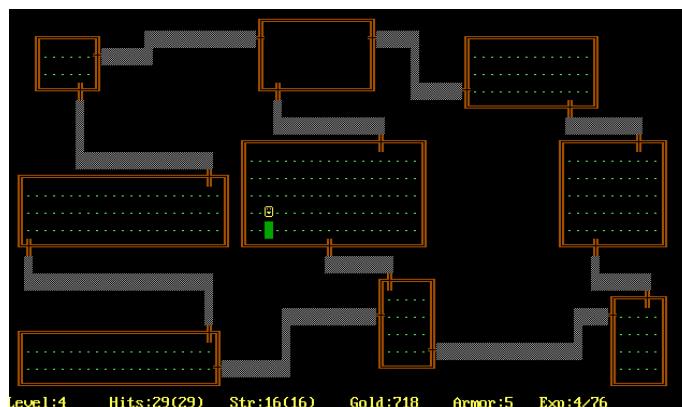


Fig. 4 Mapa de un nivel del videojuego Rogue. Fuente:
https://es.wikipedia.org/wiki/Rogue#/media/Archivo:Rogue_Screen_Shot_CAR.PNG

Un juego de 1980 también es Maze Craze, un simple juego en el que se genera aleatoriamente un laberinto y en el cual el jugador ha de escapar y había dos policías que recorrían el laberinto aleatoriamente, de color rojo y azul, los cuales tenías que eludir ya que no te debían capturar. Simple y sencillo, no se podía esperar más de la época, pero ofrecía una variedad de mapas gracias a la generación procedural.[5]

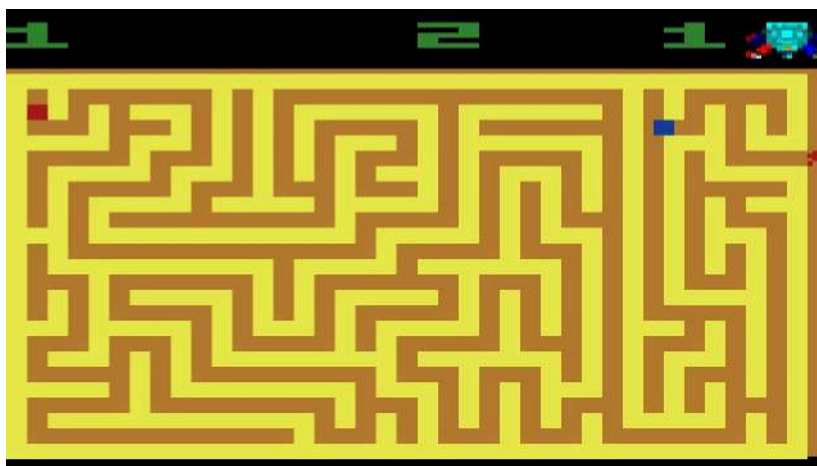


Fig. 4 Laberinto generado del juego Maze Craze. Fuente:
https://www.youtube.com/watch?v=C5mX2Viz6y8&ab_channel=OldClassicRetroGaming

Un juego que usaba fractales para generar proceduralmente y en tiempo real montañas en un planeta alienígena, es Rescue on Fractalus, donde tenías que ir con una nave a recoger unos recursos que aparecían aleatoriamente conforme se creaban las montañas y los enemigos alienígenas.[6]

River Raid utilizó muy inteligentemente una secuencia de números pseudoaleatorios generada por un registro de desplazamiento con retroalimentación lineal para obtener un laberinto lleno de obstáculos, ese registro de desplazamiento ayudó notoriamente al desarrollo debido a la limitación de hardware de la época.[7]

0	00	6	30																
1	08	7	38																
2	10	8	40																
3	18	9	48																
4	20	!	50																
5	28	null	58																
				3	4	5	6	7	8	9	a	b	c	d	e	f			
				9f	ff	00	ff	00	07	00	00	1e	00	00	45	c5		^	
				05	00	00	01	04	06	04	00	00	35	b5	50	00			
				07	0a	05	07	07	07	23	23	23	58	58	5c	00			
				4c	00	fe	01	ff	ff	0f	bb	fc	07	01	14	a8			
c	18	fb	01	14	a8	58	30	0f	fe	0f	fe	49	ff	58	fb	58			
d	fb	58	fb	58	fb	58	fb	00	fb	07	fd	58	fc	58	00	58			
e	00	58	ff	58	ff	58	ff	00	ff	4e	81	14	a8	00	d6	1c			
f	06	1e	e9	00	00	51	00	01	00	00	00	00	ff	ff	fa	f3	v		

Fig. 5 Secuencia que genera el inicio de la partida y donde se puede apreciar que podían jugar hasta dos jugadores al mismo tiempo. Fuente: https://www.youtube.com/watch?v=CT9jFKO99fE&t=158s&ab_channel=GameDevBoss

Otro ejemplo de los primeros juegos procedurales que podríamos tener, sería el juego Akalabeth, en este juego lo primero que se te pregunta es por un número mágico, y ese número es el que servirá para crear prácticamente el juego entero, los atributos, mapa de las zonas salvajes, mazmorras... Por lo que, al escribir ese mismo número, siempre podrás volver a esa generación del mundo.[8]

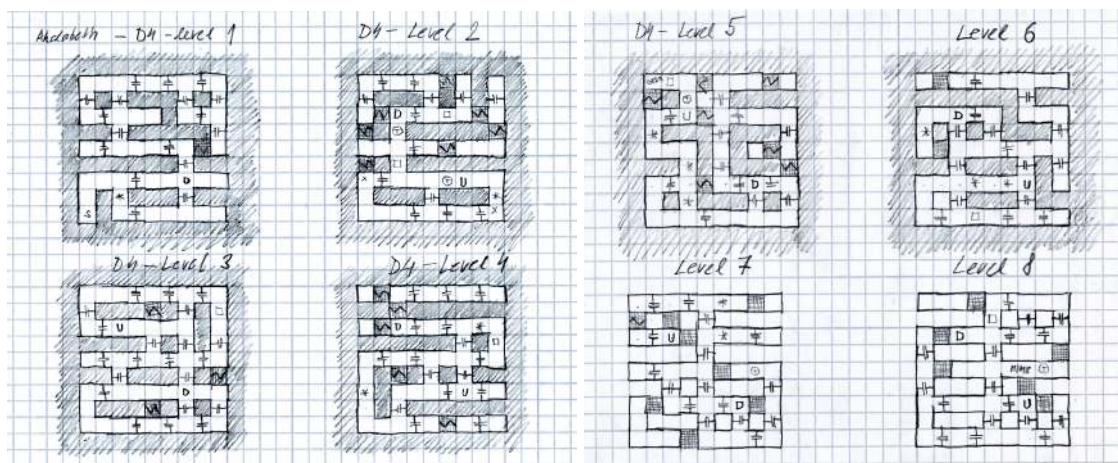


Fig. 6 Imágenes de un usuario que mapeó los diferentes niveles con el número mágico 7 y la dificultad 1. Fuente: <https://www.oldgames.sk/disc/akalabeth-dennik-hraca/>

Otro juego importante es Elite, ya que fue creado para tener 2^{48} galaxias, lo cual al final no ocurrió, pero no porque no pudiesen contener esa capacidad, si no porque el creador pensó que causaría incredulidad entre la población, pero eso no quita el hecho de que se usó la generación procedural para contener esa magnífica cantidad de galaxias en un sólo videojuego que salió a la luz en 1984, y que sirvió de influencia para los juegos venideros de esa temática, como Eve Online.[9]

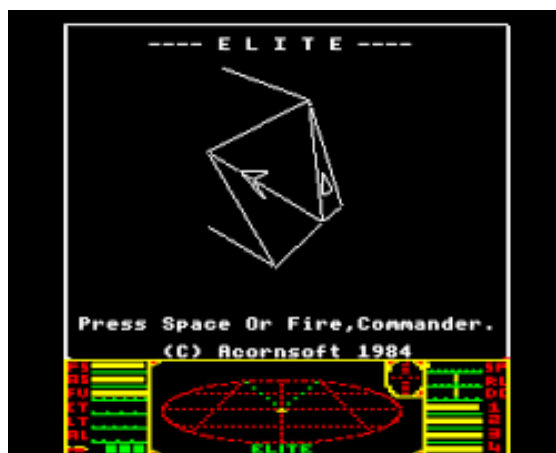


Fig. 6 Visualización del juego Elite. Fuente:
<https://www.telegraph.co.uk/technology/video-games/11051122/Elite-the-game-that-changed-the-world.html>

Conforme mejoraba el hardware y el almacenamiento de los CDs, ya no era necesario usar esta generación procedural para construir los mundos, y tanto las texturas, modelados y medio ambiente eran creados por artistas de antemano, para una mayor calidad del contenido.

Lógicamente, al hacer todo a mano, existirá un número limitado de objetos, pero al menos esos objetos presentarán un detalle óptimo. Han habido casos en los que se intentó que los programadores creasen código que generase automáticamente diferentes mallas de acuerdo a los parámetros requeridos de entrada, para así mejorar en cuestión de tiempo a los diseñadores.[10]

Pero en lugar de hacer que sólo construya contenido procedimentalmente, es más barato y eficaz que el artista cree el contenido para algunos detalles, como en SpeedTree que es un programa para generar árboles procedimentalmente, pero las texturas se pueden extraer de archivos regulares, para que parezca follaje real. Otro método eficaz sería fusionar procedimentalmente varios archivos preconfeccionados, o aplicar con esta técnica distorsiones.[11]

2.3. Actualidad

En 2004 .kkrieger, un juego en 3D de disparos en primera persona, salió a la luz, y lo sorprendente de este juego es que sólo ocupaba 96 KiloBytes, lo cual es muy impresionante para la época.

Si que es cierto que era una beta, y de hecho, a día de hoy, sólo existe esa beta, que no dura mucho tiempo, unos 7 a 10 minutos, pero sigue siendo un juego en 3D, y que ocupase tan poco lo hacía especial, y eso se consiguió gracias a la generación procedural.[12]

Al contrario que este juego, muchos otros se tenían que lanzar en DVDs que contuviesen una memoria superior a 2 GigaBytes, es decir, 20.000 veces más grande.

Pero también salieron juegos que, gracias a la generación procedural, se pudo maximizar el contenido del archivo descargable para que ocupase menos de 50 MegaBytes, como ocurre en RoboBlitz, un juego de acción y puzzles.[13]



Fig. 7 Comparativa entre .kkrieger(izquierda) y RoboBlitz(derecha), 96KB vs 50MB. Fuentes:
<https://oldgamesdownload.com/kkrieger-chapter-1/>
<https://store.steampowered.com/app/4300/RoboBlitz/?l=espa%C3%B1a>

Dwarf Fortress, también de este año 2006, es un juego que sólo muestra caracteres ASCII, y el cual genera proceduralmente todo el mundo de fantasía, tanto las criaturas, como los NPCs y la historia generada aleatoriamente. Por lo cual fue alabado en su día, por su profundidad y complejidad, pese a su escaso arte.

Tan sólo una partida podía ser jugada por mundo a la vez, en la creación del juego, el jugador podía ajustar ciertos parámetros como el tamaño del mapa, la dificultad, la probabilidad de aparición de los minerales, la duración de la historia... Y así el mapa te enseñaba los símbolos ASCII que representan los caminos, colinas y ciudades de las civilizaciones, e iban cambiando conforme progresaban.

El sistema genera de forma procedural también elementos como la altitud, precipitaciones, drenaje y temperatura, por lo que un área donde llueva mucho pero haya bajo drenaje, se

convierte en un pantano. Y así se generan los distintos biomas, también definidos por la dificultad y un sistema de moral (bondad/maldad), todos los biomas con su propia flora y fauna. Posteriormente, se simula la erosión y se crean los ríos que desembocan en océanos o lagos, y un campo de salinidad define si son océanos, manglares o llanuras aluviales.

Incluso los nombres están generados basados en el propio sistema de moral, y de uno de los cuatro idiomas del juego que existen, ya que existen las razas de los enanos, elfos, humanos y goblins. En 2019 se lanzó una versión del juego mejorada respecto a los gráficos.[14]



Fig. 8 Comparativa entre la versión ASCII y la mejora gráfica posterior. Fuente: <https://techcrunch.com/2019/03/13/the-legendary-and-indescribable-dwarf-fortress-goes-non-ascii-and-non-free-for-the-first-time/>

Pasando ya a 2008, un juego de disparos en primera persona de nuevo, pero mucho más conocido, Left 4 Dead que utiliza la generación procedural como mecánica base del juego, lo que hace este juego es que tiene una inteligencia artificial, conocida como "director" la cual analiza las estadísticas de los jugadores y los estados del juego, para darle al usuario una experiencia dinámica en cada una de las partidas.[15]

Esta IA director, lo que hacía era crear o eliminar enemigos o elementos, dependiendo de tus estadísticas de salud, munición, etc, haciendo así que cada partida se adapte al jugador y mantenga un ritmo emocionante y así aumentar el valor de la rejugabilidad.

La secuela de este juego, Left 4 Dead 2, lanzado apenas un año después del primero, a finales 2009, introduzco mejoras en esta IA, como la capacidad de generar nuevos caminos para los jugadores dependiendo de sus estados individuales y también cambia el clima de forma radical.[16]

En este mismo año 2008 salió Spore también, el cual usaba la generación procedural para muchos componentes del juego, como el diseño de criaturas, vehículos y edificios, las animaciones, texturas, colores, la generación de planetas y ecosistemas, flora y fauna, las

civilizaciones... Lo cual le proporciona al jugador un amplio alcance y un final abierto, y sobre todo, múltiples formas de poder jugarlo.[17]

Caves of Qud tiene un sistema de misiones en las que algunas están predefinidas y otras están generadas proceduralmente, por lo que el jugador puede seguir la historia principal, o por el contrario ver que le puede deparar el juego con una historia no predefinida, incluso puede empezar la historia en la ciudad principal o en una generada proceduralmente.[18]



Fig. 9 Comparativa entre la versión ASCII y el cambio de gráficos. Fuente: https://www.youtube.com/watch?v=gorwik2kHyE&ab_channel=Blind

Un juego mundialmente conocido, y que no podía faltar, es el Minecraft, el cual utiliza el ruido perlin y el ruido rosa entre otros algoritmos, para hacer la creación del mundo, y consiguiendo así que el mundo sea inmenso llegando a tener en la actualidad hasta 30 millones de bloques en todas direcciones.[19][20]

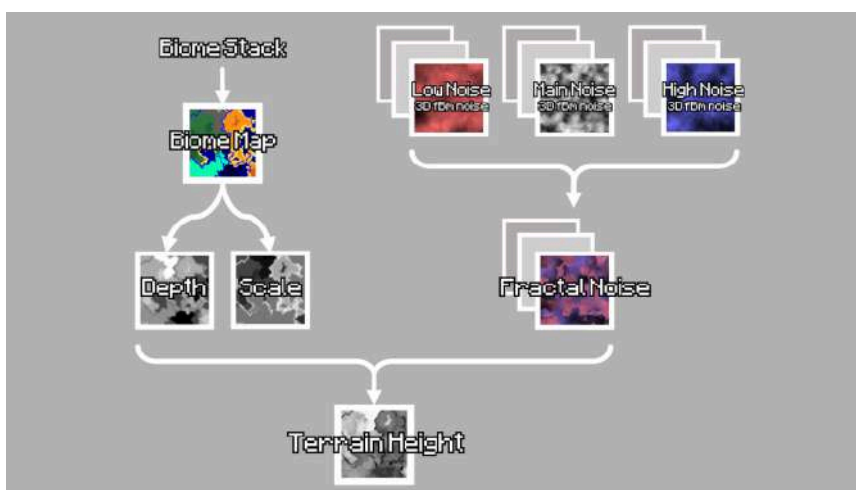


Fig. 10 Cómo se genera la altura del mundo en el Minecraft. Fuente: <https://www.alanzucconi.com/2022/06/05/minecraft-world-generation/>

Otro juego que es indispensable, es The Binding of Isaac, juego de 2011 que perdura hasta el día de hoy y se mantiene como muy buen juego, siendo el resultado de una game jam de una semana, genera proceduralmente la mazmorra con sus salas y enemigos, y ha sido inspiración para muchos otros juegos que han venido a lo largo de los años.[21]



Fig. 11 Mapa de The Binding of Isaac con las distintas zonas generadas proceduralmente. Fuente: <https://es.pinterest.com/pin/1001558404628026109/>

En 2014, 30 años después de la primera entrega de Elite, mencionada anteriormente, sale a la luz Elite: Dangerous, la cual mediante generación procedural está generando nuestro sistema solar, ya que busca crear a escala 1:1 la Vía Láctea. Y es un MMORPG en el que las acciones de los jugadores afectan directamente al mundo o universo persistente.[22]

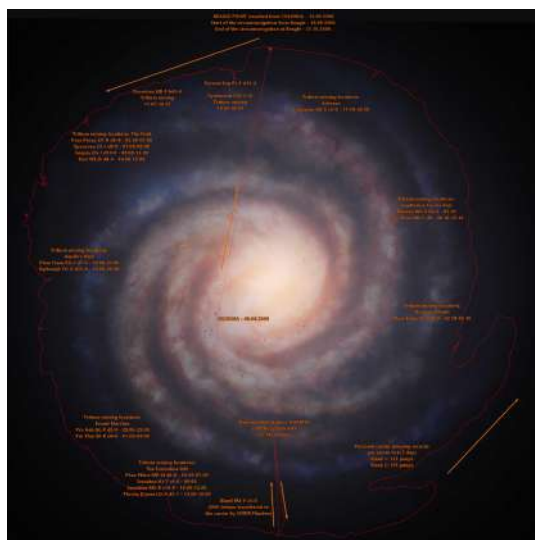


Fig. 12 Recorrido de un jugador que tardó 4 meses en dar la vuelta a la vía láctea. Fuente: <https://forums.frontier.co.uk/threads/record-primer-cmdt-en-circunnavigar-la-galaxia-en-un-porta-naves.557560/>

No Man's Sky, publicado en 2016, es un juego de exploración de planetas también, generado proceduralmente, puede albergar hasta 18 trillones de planetas posibles según los propios desarrolladores, tanto los planetas como el clima, fauna y flora, están generados por una semilla generada aleatoriamente en su motor, lo cual permite que cualquier jugador con esa semilla y la ubicación del planeta en esa galaxia virtual, pueda ser visitado por varios jugadores, el motor usa unas fórmulas matemáticas con las que se determinan la edad y disposición de las estrellas, la agrupación de los cinturones de asteroides, planetas, la física de la gravedad... Todos los parámetros para crear los diferentes planetas y galaxias.[23][24][25]



Fig. 13 Jugador se queda atrapado sin combustible y sin manera de salir de la nave en su mundo permadeath y pide ayuda en un foro de reddit, actualmente aún sigue ahí, aunque se intentó un rescate sin éxito. Fuente: <https://vandal.elespanol.com/noticia/1350780533/la-comunidad-de-no-mans-sky-ayudo-a-escapar-a-un-jugador-atrapado-en-un-planeta-extremo-durante-meses/>

A pesar de que se suele identificar a la generación procedural en los videojuegos con la generación de mapas, o la aleatoriedad de los objetos, o la animación de los personajes, hay casos en los que ha sido utilizada para crear situaciones diferentes, eventos y narrativas, como por ejemplo en Rimworld, un juego de gestión de colonias, pero hace que te puedas encariñar de los personajes de verdad, y no sentir que son simples NPCs a los que mandas a hacer una acción u otra y ahí se acaba la historia, si un personaje tiene un familiar, y de repente por algún factor, se le muere, este personaje podrá ponerse a llorar, o a colapsar mentalmente, como lo haría una persona normal y corriente, por lo que las historias dependen de las acciones que tomes como jugador, y nunca estarán predeterminadas.[26]

Otro caso similar es el juego de Moon Hunters, donde se van creando mitos conforme juegas, es decir, lo que hagas en tu primera partida y el cómo lo hagas, tendrá un efecto inmediato en la siguiente. Como por ejemplo, si ayudas a una aldea, puede que en la siguiente partida, al pasar por ahí, veas tributos o estatuas dedicadas a tu antiguo personaje, o si traicionas a algún grupo o persona, puede que conviertas en villano a alguien debido a esa acción que realizaste anteriormente.[27]

Starbound, un juego de galaxias pero esta vez en 2D, los planetas son generados proceduralmente con características únicas, usa un sistema de semillas o coordenadas generadas con el mundo, y estas coordenadas son compartidas, así que pueden visitar tu planeta otros jugadores si conocen las coordenadas, los planetas generan proceduralmente el terreno, tipos de árboles, clima, los ciclos de día y noche, gravedad, apariencia, comportamiento de los enemigos, recursos... Todo generado proceduralmente, incluso las armas y armaduras. Tiene una estética similar a la del Terraria, mencionado anteriormente.[28]



Fig. 14 Comparación gráfica y estética del Terraria(izquierda) y el Starbound(derecha). Fuente: <https://shockbyte.com/blog/starbound-versus-terraria>

Against the Storm es un juego de construcción de ciudades, en el cual se tiene que recolectar recursos para convertirlos en alimento o combustible, pero los mapas, población y edificios disponibles se generan de forma procedural, por lo que las tareas asignadas a tu aldea pueden verse complicadas por la falta de un fácil acceso a los recursos, y entonces los aldeanos pueden abandonar la aldea.[29]

2.4. Algoritmos

Existen muchas posibilidades a la hora de crear juegos con generación procedural, los principales algoritmos utilizados comúnmente son los siguientes:

2.4.1. Ruido Fractal

Los fractales se usan generalmente para la generación de mapas en tiempo real, con la ventaja de que es fácil generar mapas creíbles y además son eficientes.

Sin embargo, una limitación importante a comentar, es que sólo se usa una semilla que se genera aleatoriamente, por lo que cualquier pequeño cambio en la semilla, hará que se cree un mapa totalmente diferente.

Normalmente se desea que un algoritmo de generación procedural se pueda controlar, para especificar parámetros como el tamaño, frecuencia y distribución de las diferentes características que pueda tener el mapa final.[30]

Hay una variante para crear terrenos donde se usa la técnica de divide y conquista, es decir, recursivamente se va dividiendo un terreno cuadrado en cuatro subcuadrantes desplazando el punto central mediante algún criterio, esto se hace hasta que se alcance el nivel de detalle deseado. El tipo de desplazamiento dependerá del método de desplazamiento del punto medio que se utilice.

Se comienza con cuatro puntos de altura aleatoria en las esquinas, y se van insertando nuevos vértices en los bordes y el centro, así se puede trabajar sobre una nube de puntos para crear la altitud y desnivel del mapa, hablando en un entorno 3D, aunque también se puede pasar al entorno 2D haciendo que esa altitud significa que es posible andar por ahí, y lo que está debajo de ciertos puntos que no se puede.[31]

Para ejemplificar, en la Fig.15 se puede observar en la parte superior el ruido simple fractal en 2D y en la parte inferior en 1D, lo que ocurre es que se va añadiendo un ruido simple más en cada fase, y así se ve la diferencia de cómo puede variar, en la parte del medio se ha sumado el de la primera, y en la última los anteriores.

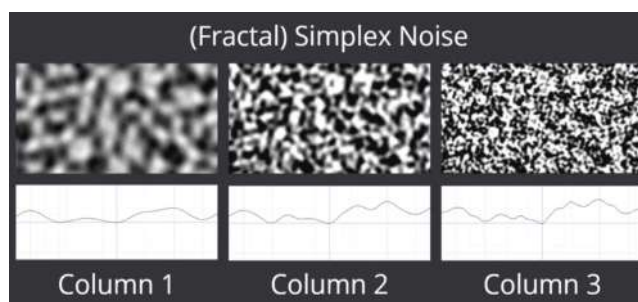


Fig. 15 Observación cuanto más ruido se añade. Fuente:
https://www.youtube.com/watch?v=Z6m7tFztEw&ab_channel=WhiteBoxDev

Para poder hacerla más aleatoria, se puede modificar la malla del ruido simple variando unos determinados parámetros como la profundidad, amplitud, frecuencia, brecha y persistencia, en la Fig.16 se puede observar cómo al cambiar los parámetros, cambia el resultado significativamente.[32]

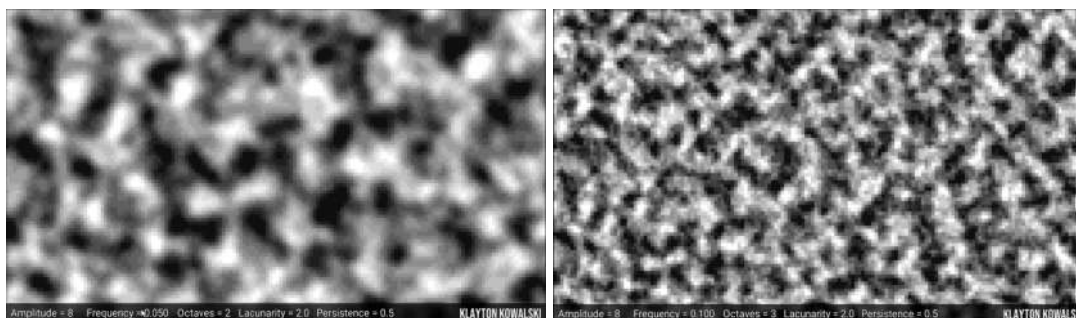


Fig. 16 Observación al cambiar los parámetros especificados. Fuente:
https://www.youtube.com/watch?v=Z6m7tFztEvw&ab_channel=WhiteBoxDev

2.4.2. Autómatas celulares

Los autómatas celulares surgen en la década de 1940 con John Von Neumann, el cual intentaba modelar una máquina que fuese capaz de autoreplicarse, inicialmente el modelo matemático se interpretó como un conjunto de células que crecen, se reproducen y mueren, y de ahí viene su nombre.[33]

Este concepto sentó las bases para autómatas posteriores, como el famoso Juego de la Vida de John Conway en 1970, que simplificó las reglas pero mantuvo la capacidad de generar patrones complejos.[34]

Son un modelo matemático compuesto por un conjunto de celdas con diferentes estados o valores, los cuales son alterados en intervalos regulares par que ese conjunto de células evolucionen según la expresión matemática, llamada también la regla de transición local.

Vienen definidos por una serie de elementos, los cuales son una cuadrícula n-dimensional de celdas, un conjunto finito de estados para cada una de esas celdas y un conjunto de reglas de transición.

El número de configuraciones posibles es igual al número de estados posibles para una celda, elevado a la cantidad de celdas. Por lo que para grupos pequeños de vecinos, lo más común es hacer una tabla asociativa con las configuraciones posibles del estado futuro, pero para grupos más grandes, las reglas se suelen basar en la proporción de celdas en cada estado. Por ejemplo, con la vecindad de Moore se tendrían $(2r + 1)^2 - 1$ vecinos, siendo "r" la unidad de distancia, es decir, que con un radio de 3, se tendrán 48 vecinos y 3 estados diferentes, por

ejemplo, tierra, agua, aire. Eso nos deja que hay 3^{48} configuraciones posibles, que son 7.9×10^{22} configuraciones posibles.[35]

Las celdas evolucionan en pasos discretos de tiempo, y su próximo estado es determinado por la regla de transición que considera su estado actual y el de sus vecinos del paso anterior.

Algunos ejemplos de reglas para hacer autómatas celulares sería la regla exactamente 1, la cual es poner un 1 en la tabla sólo si existe un único 1 a su alrededor. Por lo que si nuestra tabla es: 0 1 1 0 1 0 0 0, pasará a ser 1 1 1 0 0 1 0 0.

Analizado paso para que se pueda entender mejor el resultado y enumerando las celdas de 0 a 7 para poder explicar mejor los pasos:

Celdas: 0 1 2 3 4 5 6 7

Inicio: 0 1 1 0 1 0 0 0

Celda 0: Sólo tiene **un 1** de vecino (a su derecha), por lo que pasa a ser **1**

Celda 1: Sólo tiene **un 1** de vecino (a su derecha), por lo que se mantiene en **1**

Celda 2: Sólo tiene **un 1** de vecino (a su izquierda), por lo que se mantiene en **1**

Celda 3: Tiene **dos 1** de vecinos (izquierda y derecha), por lo que se mantiene en **0**

Celda 4: No tiene **ningún 1** de vecino, por lo que pasa a ser **0**

Celda 5: Sólo tiene **un 1** de vecino (a su izquierda), por lo que pasa a ser **1**

Celda 6: No tiene **ningún 1** de vecino, por lo que se mantiene en **0**

Celda 7: No tiene **ningún 1** de vecino, por lo que se mantiene en **0**

Final: 1 1 1 0 0 1 0 0

Este autómata celular, si se siguieran haciendo pasos, quedaría un patrón como se muestra en la Fig.17 y lo cual es un triángulo de Sierpinski, que es una figura fractal, pero si modificamos la condición inicial, en vez de tener algo tan ordenado, se puede tener una formación más compleja y aleatoria, como se aprecia en la Fig.18.

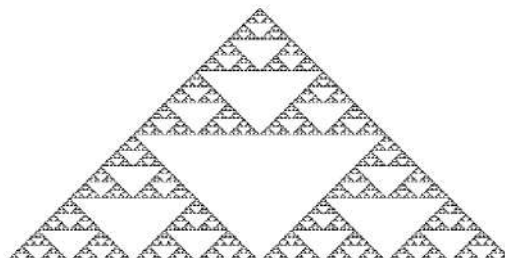


Fig. 17 Patrón con la regla exactamente 1 con la sucesión 01101000. Fuente: https://www.cs.us.es/~fsancho/Blog/posts/Automatas_Celulares.md.html

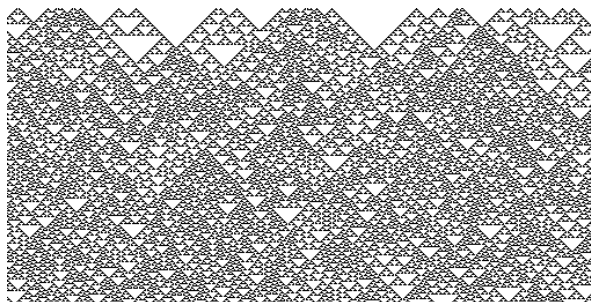


Fig. 18 Patrón con la regla exactamente 1 con un valor inicial diferente. Fuente: https://www.cs.us.es/~fsancho/Blog/posts/Automatas_Celulares.md.html

Hay muchas más reglas para hacer autómatas celulares, sólo depende de las reglas que quieras poner, y así saldrán patrones totalmente diferentes, la regla de la mayoría, la de solidificación, puedes poner más estados aparte de 0 o 1, puedes poner un 2 en determinadas ocasiones que predefinas y eso dará un patrón diferente con cada uno.

Se pueden generar patrones extremadamente complejos con unas reglas muy simples y lo mejor es que son fáciles de definir desde un punto de vista matemático y de representar con ordenadores. Por contraparte, con muchos vecinos puede llegar a ser un problema computacional, como con la vecindad de Moore.

Este algoritmo se ha utilizado sobre todo para modelar sistemas ambientales, como el calor, fuego, lluvia, explosiones... y en combinación con mapas para la toma de decisiones de agentes. También para simular la erosión térmica e hídrica en la generación procedural de terrenos.[36]

2.4.3. Ruido Perlin

El Ruido Perlin se ha utilizado para la creación de mapas en juegos de construcción de reinos, comparar su eficacia respecto al ruido simple, juegos de exploración tipo sandbox o generar islas en juegos de mundo abierto. Aunque también se ha usado en la aplicación de cambios pseudoaleatorios a variables y en la creación de texturas para imágenes para aumentar el realismo en gráficos por computadora.[37]

Este algoritmo fue desarrollado por Ken Perlin en el año 1983, y en el año 1985 documenta en su artículo "An Image Synthesizer" cómo diseñar imágenes generadas por computadoras.[38]

Lo que permite es hacer múltiples copias escaladas que pueden insertarse en expresiones matemáticas para generar una variedad de texturas procedurales.[39]

Este ruido se suele implementar como una función de 2 a 4 dimensiones, aunque se puede definir para el número que se desee.

La implementación más típica es definir una cuadrícula de vectores gradientes, calcular el producto escalar entre los vectores y sus desplazamientos e interpolar los valores.

En la Fig.19 se puede observar el primer paso, que es la generación de una cuadrícula n-dimensional en la cual cada intersección tiene asociado un vector de gradiente, el cual es de longitud unitaria aleatoria n-dimensional.

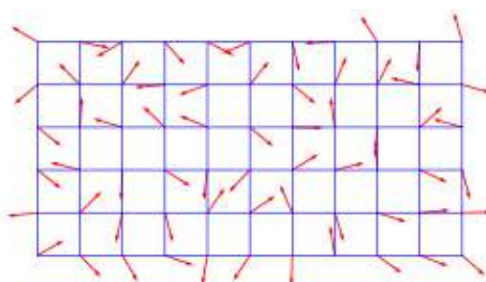


Fig. 19 cuadrícula bidimensional de vectores de gradiente. Fuente: https://en.wikipedia.org/wiki/Perlin_noise#cite_note-gustavson-7

El siguiente paso es calcular el producto escalar, y para ello hay que calcular el valor de los puntos. Hay que saber en qué celda de la cuadrícula está, identificar los vértices de esa celda junto con los vectores de gradiente asociados, y calcular el vector de desplazamiento desde ese vértice hasta el punto candidato. Esto hay que hacerlo para cada esquina, por lo que este algoritmo tiene una complejidad de $O(2^n)$.

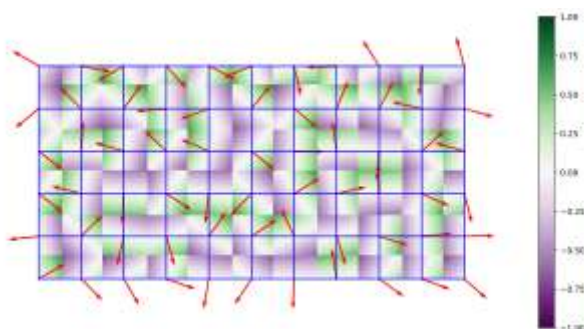


Fig. 20 Resultado del producto escalar de la figura anterior. Fuente: https://en.wikipedia.org/wiki/Perlin_noise#cite_note-gustavson-7

Y por último ya es interpolar entre todos los puntos, para ello se usa una función que la primera derivada y posiblemente la segunda, es 0 para todos los nodos, haciendo así el aspecto característico de este ruido, como podemos ver en la Fig.21, que hace que se vea más fluido y no que parezca dividido en cuadrados.

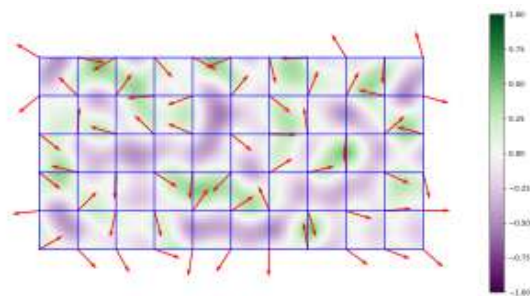


Fig. 21 Resultado de la interpolación. Fuente: https://en.wikipedia.org/wiki/Perlin_noise#cite_note-gustavson-7

Para poner un ejemplo más visual y con datos, una forma de hacer este ruido es la siguiente:

- Partimos de la malla de puntos y les asignamos sus posiciones

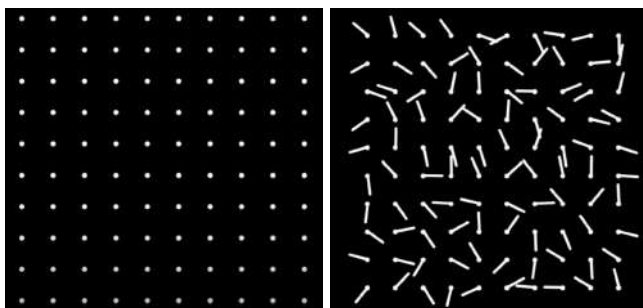


Fig. 22 Cuadrícula de puntos y sus direcciones. Fuente: https://www.youtube.com/watch?v=9B89kwHvTN4&ab_channel=Doggo%27sScience

- Después nos centramos en 4 de estos puntos de los cuales ya sabemos la dirección que llevan y cogemos un punto aleatorio entre ellos:

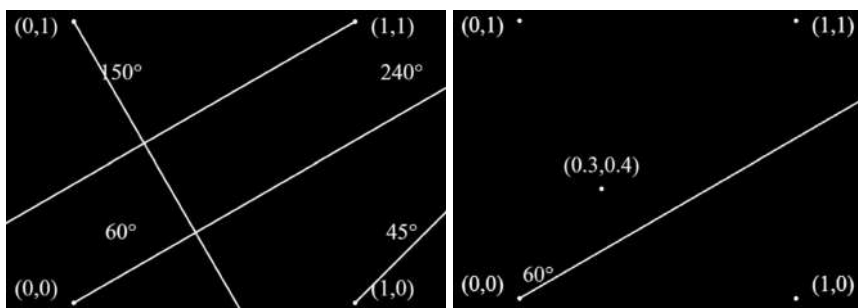


Fig. 23 Los cuatro puntos con sus direcciones y el punto aleatorio entre los puntos. Fuente: https://www.youtube.com/watch?v=9B89kwHvTN4&ab_channel=Doggo%27sScience

- Ahora que ya tenemos el punto, es el momento de hacer el producto escalar, es decir, en este caso, en el punto (0,0), tendríamos: $(-0.3 \cdot \cos(60)) + (-0.4 \cdot \sin(60)) \approx -0.496$, y habría que repetir el mismo proceso con los 4 puntos, y saldría algo como en la Fig.24.

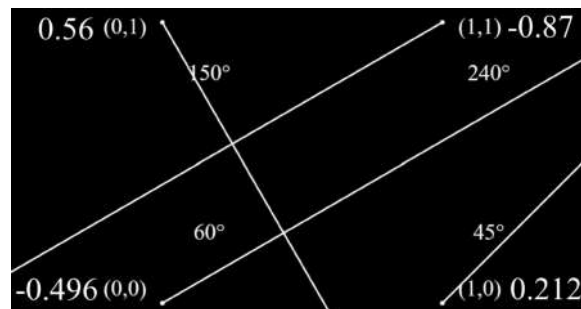


Fig. 24 Los cuatro puntos con sus direcciones y el punto aleatorio entre los puntos. Fuente:
https://www.youtube.com/watch?v=9B89kwHvTN4&ab_channel=Doggo%27sScience

- Ahora ya solamente queda interpolar, y una vez interpolado en cada punto, quedará lo que se puede apreciar en la Fig.25

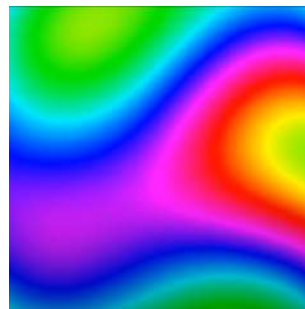


Fig. 25 Resultado de la interpolación. Fuente:
https://www.youtube.com/watch?v=9B89kwHvTN4&ab_channel=Doggo%27sScience

Por lo que si ahora continuamos haciendo eso a lo largo de la cuadrícula, nos quedará el resultado deseado del ruido Perlin.

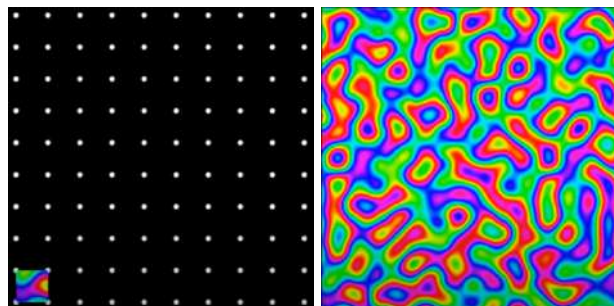


Fig. 26 Resultado final haciendo la implementación en cada cuadro. Fuente:
https://www.youtube.com/watch?v=9B89kwHvTN4&ab_channel=Doggo%27sScience



2.4.4. Sistema de grafos

Hay muchas maneras de hacer un sistema de grafos para generar proceduralmente habitaciones, o mapas, o eventos... Este sistema se basa principalmente en tener una serie de puntos, los cuales puedes querer tener numerados para saber el orden o simplemente tenerlos para después enlazarlos con algún método como Kruskal, Prim, o cualquier algoritmo de la teoría de los grafos.

Ambos métodos son árboles recubridores mínimos, es decir, que buscan el camino más corto hacia el objetivo deseado, el método de Kruskal[40] funciona de la siguiente manera:

Primero se crea un conjunto de árboles donde cada vértice es un árbol, después se crea un conjunto que tenga todas las aristas, mientras ese conjunto no sea vacío se elimina una arista del conjunto, si esa arista conecta dos árboles diferentes se añade al conjunto inicial combinando ambos árboles en un sólo árbol, si no, se excluye esa arista.

Siendo "m" el número de aristas y "n" el número de vértices, este algoritmo tiene una complejidad de $O(m \log m)$ u $O(m \log n)$, ya que "m" como mucho es n^2 el $\log(n^2) = 2\log(n)$, que es $O(\log n)$, y al ignorar los vértices aislados, n es menor o igual a 2m, por lo que el $\log(n)$ es igual a $O(\log m)$.

Por otra parte, el método de Prim[41] incrementa el tamaño del árbol continuamente, empieza por un vértice al que se le van añadiendo los diferentes vértices con la distancia menor.

Básicamente hace lo siguiente:

1. Se inicia el árbol con un vértice elegido aleatoriamente
2. Se aumenta por un lado, el que menor distancia tenga y que no esté previamente unido a ningún otro vértice
3. Repetir el segundo paso hasta tener todo el grafo conectado

La complejidad de este método es variada, ya que se pueden implementar de diferentes maneras, con una lista enlazada, un vector de vértices o con una estructura de datos con una cola de prioridades.

Para ver algún tipo de sistemas de grafos, Ondřej Nepožitek, explica en un paper[42] y hace un análisis e implementación de un trabajo anterior sobre un algoritmo que organiza habitaciones como nodos y asignándoles una forma y posición asegurando que comparten frontera para unir las fácilmente.

En su implementación cambia partes de la investigación anterior para poder crear pasillos entre habitaciones, diferentes tipos de habitaciones...

Todo su trabajo lo explica más simplificado pero igual de concreto en su blog[43][44] en el cual divide en varias partes el proceso.

Con el algoritmo anteriormente mencionado, se pueden hacer todas las variaciones que el desarrollador desee, como se puede observar en la Fig. 27 se puede hacer un camino directo al jefe, o que tenga salidas a otras habitaciones por medio, o algún que otro atajo...

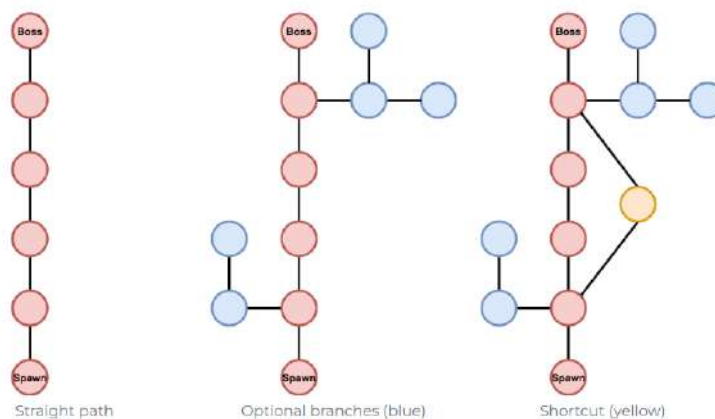


Fig. 27 Diferentes formas de generar caminos en el mapa. Fuente:
<https://ondra.nepozitek.cz/blog/graph-based-dungeon-generator-basics-1/>

Esto viene genial para poder hacer una variación de mapas, habitaciones y recorridos totalmente aleatorios para que la experiencia del jugador sea única en cada nivel o cada vez que quiera jugar al juego.

Este algoritmo a su vez genera habitaciones con diferentes formas, en las cuales puedes asignar si la sala del jefe quieras que sea de determinadas dimensiones, y las formas aleatorias de las distintas salas.



Fig. 28 Se pueden observar las distintas formas de las salas, y la sala del jefe final más grande que las demás para que se pueda diferenciar del resto. Fuente: <https://ondra.nepozitek.cz/blog/graph-based-dungeon-generator-basics-1/>

Para tener las habitaciones juntas y con las distintas formas sin que estén superpuestas, se podría hacer probando todas las posibles combinaciones, pero sería muy poco eficaz, y en lugar de ello el algoritmo calcula como se podrían conectar correctamente las habitaciones individuales, y a partir de ahí va creando el mapa.

Es más eficaz pero lo mejora aún más muy bien dividiendo el problema principal en subproblemas, descomponiendo el grafo en pequeñas cadenas y después uniendolas una por una.

En la Fig. 29 se puede ver cómo hace esa descomposición del problema para unir después el resto de salas a partir de esa base.

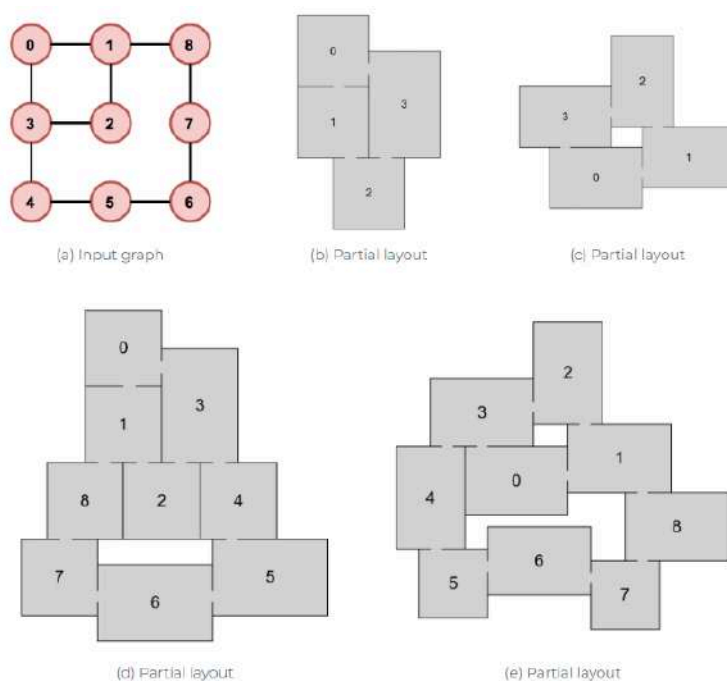


Fig. 29 Descomposición del problema y generación del mapa generado con el input del grafo. Fuente: <https://ondra.nepozitek.cz/blog/graph-based-dungeon-generator-implementation-2/>

2.4.5. Partición binaria del Espacio (BSP)

Es un método para subdividir un espacio de manera continuada en subáreas de una manera aleatoria, para así tener habitaciones que no estén superpuestas entre sí.

Toma un espacio determinado y lo divide en dos, y así recursivamente hasta alcanzar el umbral predefinido, algunos umbrales comunes incluyen el número máximo de subdivisiones permitidas o el tamaño promedio de las áreas resultantes[45].

El procedimiento del algoritmo BSP que introducen Putra, Tarigan y Zamzami en su paper consta de cuatro apartados:

- 1) Partiendo desde un área inicial denominada como "Leaf", dividirla en dos en un punto aleatorio, que serán los "leaf childs".
- 2) Repetir el proceso hasta que las áreas tengan aproximadamente el tamaño de sala deseada.
- 3) Añadir una sala dentro de cada "leaf".
- 4) Conectar las salas de los "leaf" hermanos con pasillos y continuar así hacia arriba del árbol hasta que estén todas conectadas.

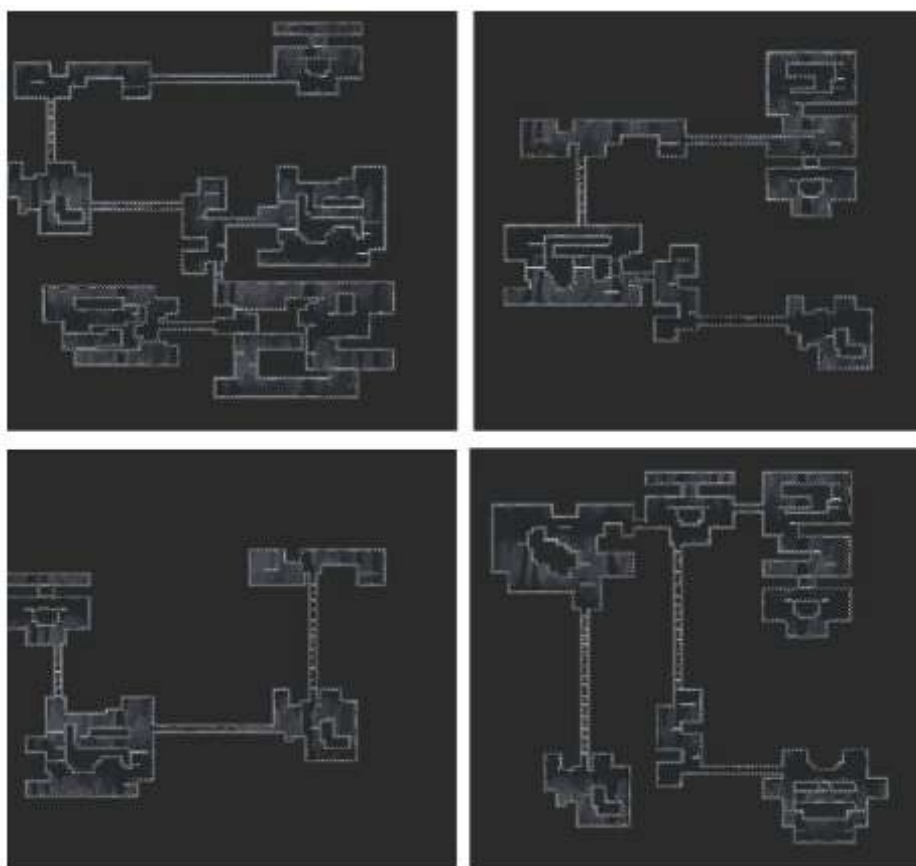


Fig. 30 Ejemplos de diferentes mapas generados proceduralmente con el algoritmo BSP mencionado. Fuente: <https://ieeexplore.ieee.org/abstract/document/10285811>

Para evitar que no haya salas idénticas primero se revisan las salas que menos veces han aparecido para ver si pueden ubicarse en un "leaf" disponible, y cuando ya están todas las salas ubicadas, se construyen los muros y se resuelven los problemas para que los muros y los pasillos hagan habitaciones coherentes.

Eso es un ejemplo de cómo aplicar el BSP en un entorno de generación de mapas, pero no se empezó a utilizar por ello, empezó siendo utilizado para un entorno de renderizado de polígonos[46] y se hacía de la siguiente manera para generarlo:

- 1) Se cogía un polígono de la lista inicial.
- 2) Se creaba un nodo en el árbol con ese polígono.
- 3) Luego para el resto de polígonos de la lista:
 - a) Completamente delante del plano → Se le añade a la lista de frente
 - b) Completamente detrás del plano → Se le añade a la lista de detrás
 - c) Si atraviesa el plano → Se divide en dos y se añade cada parte a la lista de frente o de detrás
 - d) Si está en el mismo plano → Se le añade al mismo nodo
- 4) Repetir recursivamente el proceso hasta que no haya polígonos.

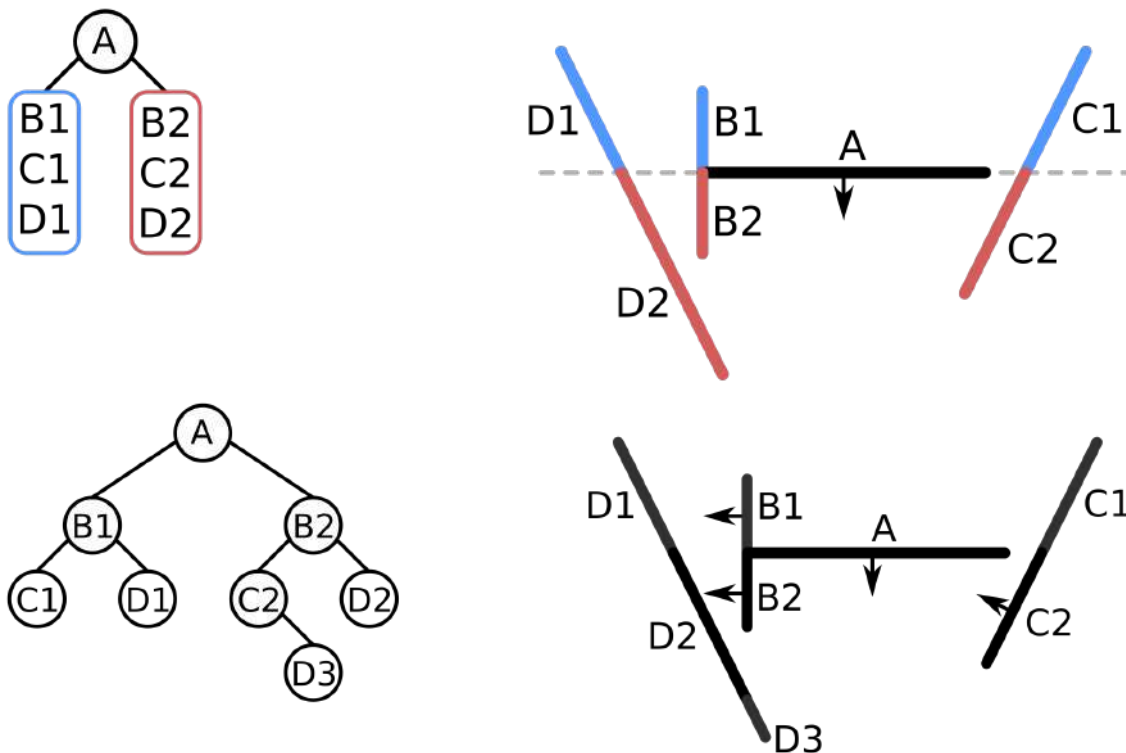


Fig. 31 Ejemplos de cómo se generaban los árboles de los polígonos, arriba el inicio donde está el plano "A" dividiendo arriba y abajo las diferentes posiciones, y abajo cuando ya se habían dividido en todas las opciones siguiendo los pasos explicados previamente. Fuente: https://en.wikipedia.org/wiki/Binary_space_partitioning

3. Objetivos

Los objetivos de este TFG son informar de la historia de la generación procedural, para saber cómo se han usado estas técnicas en los distintos juegos a lo largo de la historia, y también explicar un poco distintos tipos de algoritmos que existen para dar una idea de cómo se utilizan o cómo son para ayudar a quienes busquen inspiración o ayuda en su proyecto.

Lo principal a implementar es:

- Crear un entorno de juego en Unity 2D que utilice generación procedural para diseñar mapas únicos y funcionales en cada partida.
- Investigar e implementar distintas técnicas de generación procedural, evaluando la eficiencia de cada una y sus implicancias en la jugabilidad y rendimiento del entorno.
- Registrar detalladamente el proceso de diseño, los retos técnicos enfrentados y los resultados producidos.

Con esta implementación junto con el contexto histórico, se pretende enseñar qué se puede hacer y cómo hacerlo.

4. Metodología

Este TFG sigue una metodología analítica y experimental, dividida en diferentes fases:

1. Lo primero que busca es poner los conceptos básicos y la historia de la generación procedural, ya que es importante saber el porqué de los inicios y el cómo los diferentes juegos a lo largo de los años han ido implementando la generación procedural, ya sea para la creación de mapas, que es el pilar base de esta investigación, como en general la creación procedural en sí. Y a su vez hablar también de los distintos tipos de algoritmos que se usan en los videojuegos con una pequeña explicación de cada uno de ellos.
2. Posteriormente profundizar en los algoritmos de Grafos y de BSP con una demostración experimental en Unity en un entorno 2D, para comprobar el rendimiento de los distintos algoritmos con diversas funciones a su vez, para ver pros y contras de cada una de ellas.
3. Finalmente unas conclusiones más en profundidad de lo que se explica en el apartado anterior.



5. Implementación

5.1. Implementación Algoritmo de Grafos

Para implementar el algoritmo de grafos lo primero que he buscado son distintas técnicas que me puedan servir para hacer las funciones que me ayuden a plasmar la programación en el código, como son el algoritmo de Prim y de Kruskal.

Aparte de estos algoritmos, también tengo una función para probarlos con diferentes tipos de espaciado, para probar si influye o no con las diferentes cuentas matemáticas, o da igual el espaciado que tenga y como sea, y que sólo influya la función en sí.

Para poder comprobar el tiempo de creación de los algoritmos, al crear los nodos y sus pasillos, utilizo la clase Stopwatch para que empiece el cronómetro y que pare una vez finalizada la creación. También compruebo el tiempo de la CPU y la memoria utilizada.

El algoritmo de Prim permite encontrar el árbol recubridor mínimo, es decir, encontrar el subconjunto de aristas que formen el árbol con todos los vértices, y con el menor peso de las aristas. Es importante que todo el grafo esté conectado, de otra forma no se podrá hacer.

He implementado dos versiones de este algoritmo, para poder comprobar el rendimiento del mismo en diferentes escenarios.

La primera función que he creado es la de 'ConnectRoomsWithPrim', la cual tiene una complejidad de $O(V^2)$, que es muy eficiente para pocas salas, pero conforme se van aumentando, va perdiendo considerablemente la eficiencia. Donde V es el número de vértices totales, y se asume que todas las habitaciones pueden conectarse entre sí.

La segunda función que he creado es la de 'ConnectRoomsWithPrimOptimized', la cual tiene una complejidad de $O(E \log(V))$, y es mucho más eficiente en todos los ámbitos, pero es más complicada de hacer, ya que requiere de una PriorityQueue o en mi caso de una función Heap. En este caso ' E ' es el número de aristas y V sigue siendo el total de vértices del grafo.

En el supuesto de tener 100 habitaciones, y que cada habitación se conecta con 5, tendríamos lo siguiente:

$$O(V^2) \rightarrow 100^2 = 10.000 \text{ operaciones}$$

$$O(E \log(V)) \rightarrow 500 \times \log_2 100 \approx 3500$$

' E ' es igual a 500 porque es la multiplicación de las habitaciones totales con las habitaciones conectadas.

Una vez implementadas las funciones, empiezo con la función `ConnectRoomsWithPrim` a probar creando 10 habitaciones y con un espaciado que he llamado 'Square', el cual hace que las habitaciones sean aleatorias dentro de un área cuadrada de 20 x 20 unidades.

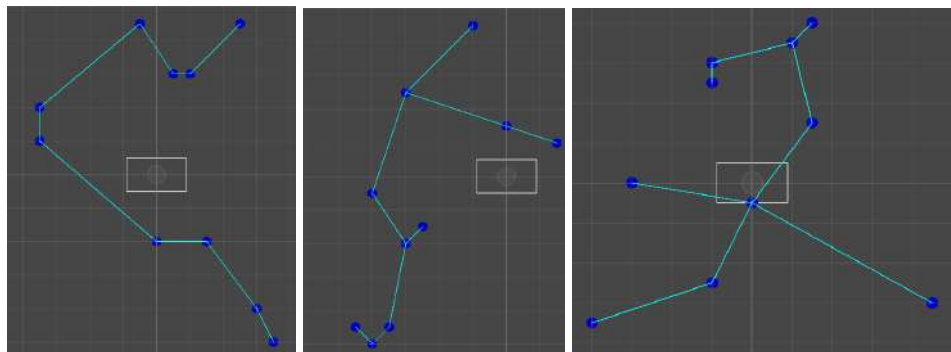


Fig. 32 Generaciones aleatorias con `ConnectRoomsWithPrim` y `Square`, 10 habitaciones. Fuente: Propia a través de Unity

Los puntos en color azul oscuro serían las habitaciones, y las líneas azules que conectan el grafo, serían los pasillos. Con estos valores, el tiempo de ejecución es de 0,057 milisegundos de media, el tiempo de CPU de 0,068 milisegundos y 0 KB de memoria usada.

Aparte del espaciado `Square`, tengo diferentes espaciados más:

- `SquareRandom`: Es parecido al `Square`, pero en vez de un 20 x 20 es un 100 x 100, sólo que en las zonas del centro hay más densidad que en las zonas exteriores
- `Pentagon`: Genera un punto aleatorio dentro de un círculo de radio 10, rotado en múltiplos de 72°, con mayor densidad de puntos cerca del centro.
- `Circle`: Parecido al `Pentagon`, pero haciendo una forma más circular.

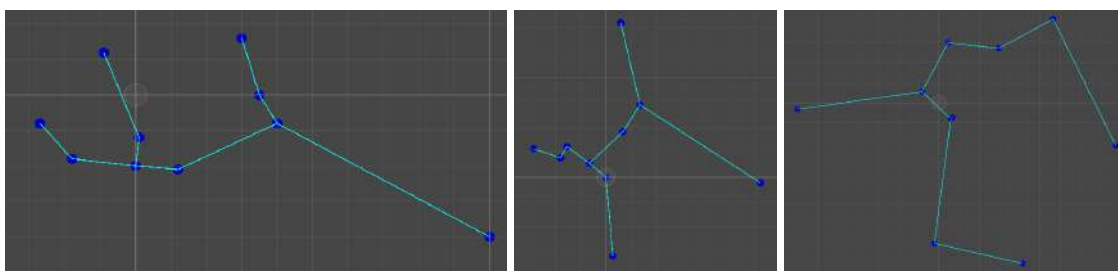


Fig. 33 Generaciones aleatorias con `ConnectRoomsWithPrim` y `SquareRandom`, `Pentagon` y `Circle`, en ese orden de izquierda a derecha, 10 habitaciones. Fuente: Propia a través de Unity

Varía poco de forma pese a estar creadas con espaciados diferentes, debido a que con este número de habitaciones no se puede ver realmente la diferencia.

Ni el tiempo de ejecución ni el tiempo de la CPU varían apenas con el resto de tipo de espaciados, así como la memoria usada.

Con el algoritmo mejorado de Prim por el contrario tarda 0,067 milisegundos la ejecución, y el tiempo de la CPU es de 0,076 milisegundos, que es algo superior al algoritmo base. La memoria usada es la misma.

Las formas tampoco cambian, como se puede observar en la Fig.34

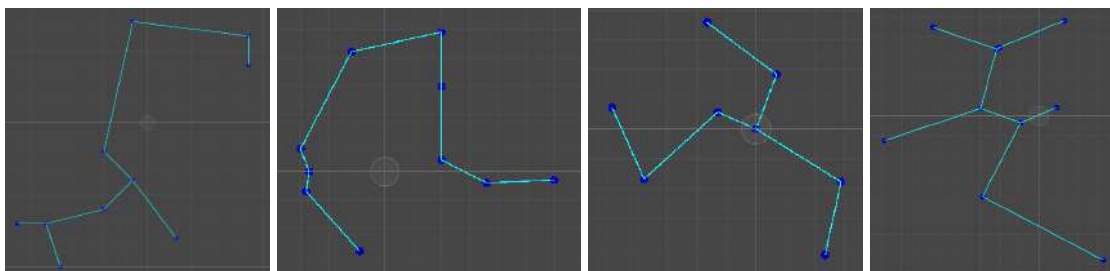


Fig. 34 Generaciones aleatorias con ConnectRoomsWithPrimOptimized y Square, SquareRandom, Pentagon y Circle, en ese orden de izquierda a derecha, 10 habitaciones. Fuente: Propia a través de Unity

El problema de ConnectRoomsWithPrim viene cuando hay más habitaciones que crear, cuando pasamos de 10 a 100 habitaciones. En general con todos los tipos de espaciado, un poco más un poco menos, el tiempo de ejecución está en los 3,8 milisegundos, el tiempo de la CPU en torno a los 4,5 milisegundos y 8 KB de memoria usada.

Con más habitaciones se puede ir viendo la forma de las distintas funciones de los espaciados, en la Fig.35 se puede apreciar en la primera imagen como tiene una forma cuadrada y se dividen por igual, mientras que en la segunda se puede apreciar esa forma cuadrada pero con más habitaciones en el centro de la cuadrícula, la tercera imagen es más aleatoria, que es lo que se buscaba haciendo esa función de estrella o pentágono, y la última más circular.

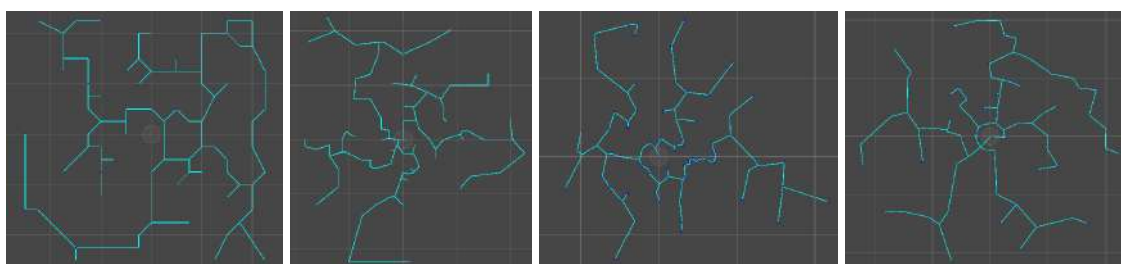


Fig. 35 Generaciones aleatorias con ConnectRoomsWithPrim y Square, SquareRandom, Pentagon y Circle, en ese orden de izquierda a derecha, 100 habitaciones. Fuente: Propia a través de Unity

Con ConnectRoomsWithPrimOptimized se puede ver una clara mejoría respecto al algoritmo anterior, en vez de tener un tiempo de espera superior, por poco que fuese, con 100 habitaciones se consigue bajar el tiempo a más de la mitad, y en estos casos sí que influye el tipo de espaciado que se usa.

Tampoco es que varíe en exceso, pero no es tan fijo como el anterior, en este caso varía entre los 1,25 y los 1,65 milisegundos de ejecución, los 1,8 y los 2 milisegundos del tiempo de la CPU y la diferencia en los KiloBytes de memoria es que en este caso son 21,6 KB de media.

Se puede comprobar también en la Fig.36 que no varían en exceso los nodos y sus conexiones con el algoritmo mejorado.

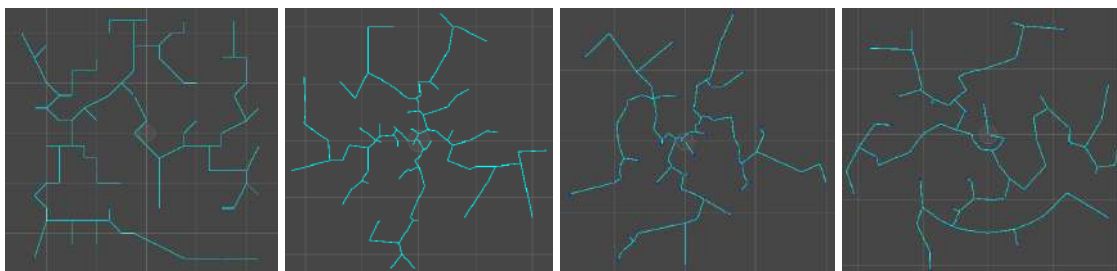


Fig. 36 Generaciones aleatorias con ConnectRoomsWithPrimOptimized y Square, SquareRandom, Pentagon y Circle, en ese orden de izquierda a derecha, 100 habitaciones. Fuente: Propia a través de Unity

El algoritmo de Kruskal es similar al de Prim, también busca encontrar el árbol recubridor mínimo y el grafo necesita estar conectado. Ordena las aristas por peso y las va añadiendo al árbol si conectan componentes distintos, al final se tendrá un árbol de expansión mínima con todos los nodos igual que en Prim en el que la suma de los pesos es la menor posible.

Con el algoritmo de Kruskal varía entre los 0,05 milisegundos de ejecución, 0,06 milisegundos de tiempo de la CPU y 0 KB entre los 4 tipos de espaciados y con 10 salas en total.

Cuando pasamos a las 100 salas nos encontramos con 1,48 milisegundos de ejecución, 1,77 milisegundos de tiempo de CPU y 5,6 KB de memoria usada.

Que está en mitad de los algoritmos de Prim, aunque más cercano al algoritmo mejorado en cuestión de ejecución y de CPU, pero se aleja mucho respecto a los KBs de memoria usada, siendo este de 4 veces menos que el método de Prim mejorado, aunque está a la par con el método de Prim normal.

Los mapas son parecidos en ambos casos de 10 y 100 habitaciones, con sus variaciones por la aleatoriedad pero no cambia nada más, lo cual es lo esperado.

En este punto, me puse a diferenciar los nodos de las habitaciones para poner las diferentes dificultades, dividiéndolas en fácil, normal y difícil, aparte de la sala inicial y la sala del boss.

También estuve revisando más a fondo diferentes juegos de este estilo de mazmorras, porque ni 10 habitaciones ni 100 habitaciones son un ejemplo realista de creación de mapas, porque 10 son muy pocas, que en algunos juegos más retro usan incluso menos, como en "Nuclear Throne", que son unas 3 habitaciones muy juntas que parecen sólo una por nivel, pero también hay otros juegos que llegan a las 50 o 150 que ya es muy grande y no suele pasar eso, un

ejemplo es el "Dead Cells", pero en general en juegos como "Enter the Gungeon" son unas 20 habitaciones, en el "Hades" de 8 a 20, "Slay the Spire" de 10 a 15 y cómo no, de los más importantes de este género, el "The Binding of Isaac" tiene de 8 a 20 salas.

Por lo que como se puede apreciar en la Fig. 37 he probado poniendo unos porcentajes de habitaciones fáciles (verde), normales (azul), difícil (morado) y la habitación principal (amarillo) y la habitación del boss (rojo).

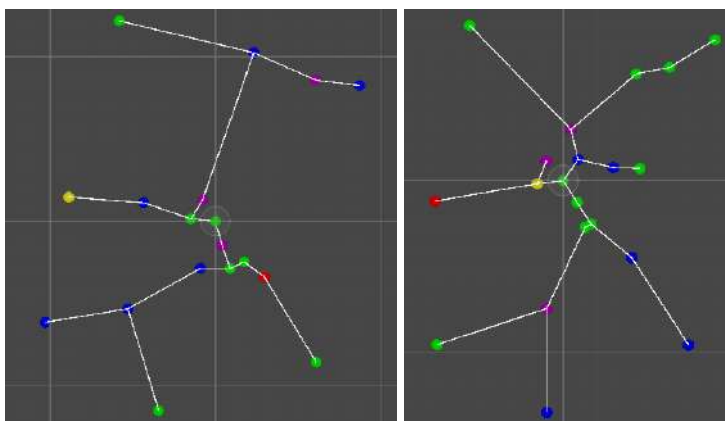


Fig. 37 Generación de habitaciones con los nuevos parámetros de creación de habitaciones. Fuente: Propia a través de Unity

Pero como se puede apreciar, la sala del boss puede estar al lado de la sala inicial, o que de a otra sala posterior y que no se puede alcanzar por otro camino, lo cual no debería pasar, ya que la sala del boss no debería dar a nada más.

Así que, ajustando el código, he puesto la sala del boss lo más alejada de la sala principal, haciendo que se asignen una vez están todos los puntos unidos.

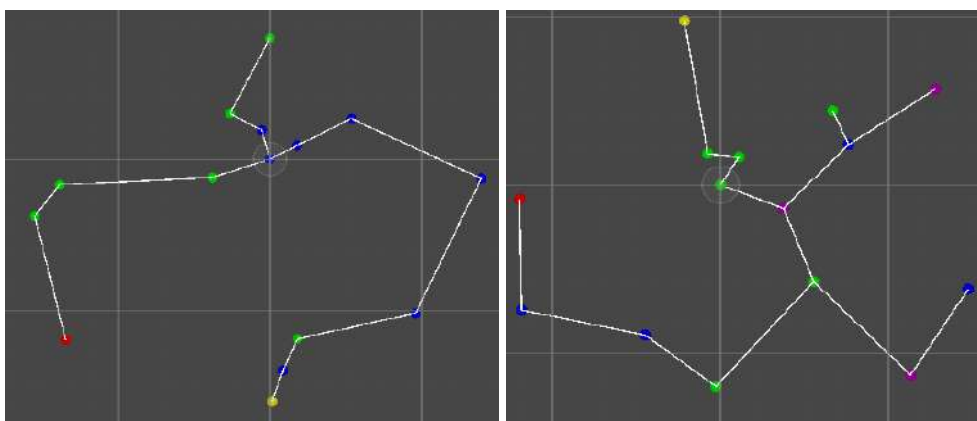


Fig. 38 Generación de las nuevas habitaciones con la sala inicial y la del boss alejada. Fuente: Propia a través de Unity

Se puede apreciar que las habitaciones en estos casos, están muy juntas, y las líneas que juntan las habitaciones no son totalmente verticales u horizontales, lo cual dificulta, que no implica que no se pueda hacer, la construcción posterior de los pasillos entre las habitaciones.

Para que sea más fácil la implementación y la generación de los mapas, me he creado una nueva función para que las habitaciones estén a la misma distancia tanto horizontal como vertical, para que así las habitaciones sean más fáciles de conectar al crear los pasillos, pero se podría hacer perfectamente la generación del mapa con los métodos usados hasta ahora, sólo que se suele estilizar más en este género de mazmorras para lo cual estoy generando los mapas, son mapas que se conectan las habitaciones de manera horizontal y vertical.

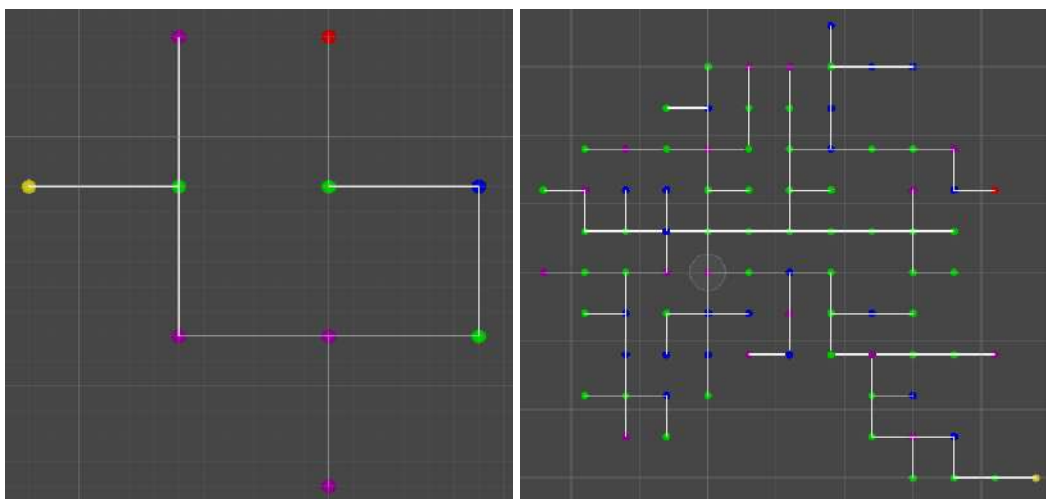


Fig. 38 Generación de habitaciones con el nuevo método para 10 y 100 habitaciones. Fuente: Propia a través de Unity

Con este método, con 10 habitaciones me da una media de 0,038 milisegundos de ejecución entre las 3 funciones de Kruskal y Prim, 0,047 milisegundos de tiempo de la CPU y 0 KB, lo cual es menor que todos los algoritmos anteriores usados con 10 habitaciones.

Respecto a hacerlo con 100 habitaciones si que voy a distinguir entre Kruskal, que son 1,4 milisegundos de ejecución, 1,68 milisegundos de tiempo de CPU y 7,2 KB.

Con el método de Prim normal, me da 3,63 milisegundos de ejecución, 4,36 milisegundos de tiempo de CPU y 5,6 KB.

Y con el método de Prim mejorado el tiempo de ejecución son 0,78 milisegundos, 0,94 milisegundos de tiempo de CPU y 24 KB.

Lo cual presenta una breve mejora con los métodos de Kruskal y Prim normal, pero una gran mejora respecto al método de Prim optimizado con los anteriores casos.

Para que sea más visual, he puesto unos sprites de colores representando la dificultad de las habitaciones, y que constaría de la amplitud de la sala, lo que he hecho es como si esto fueran las salas, que serían salas prefabricadas con sus enemigos personales y sus recompensas, pero se podría hacer de mil formas para que fuese todo aún más aleatorio.

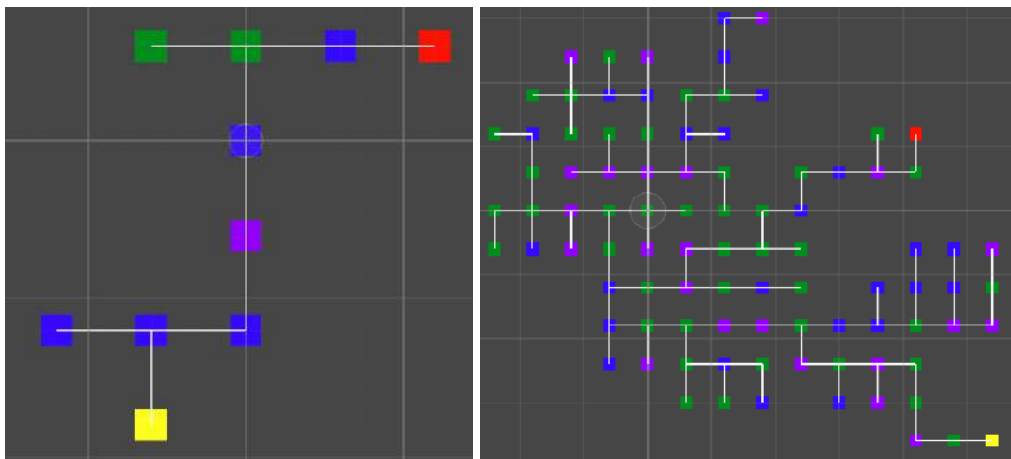


Fig. 38 Generación de habitaciones para 10 y 100 habitaciones con los sprites de las diferentes salas. Fuente: Propia a través de Unity

Lo último que falta es poner los pasillos en vez de la visualización del Gizmos, los cuales son sprites que van desde el inicio de una sala hasta el inicio de la siguiente, repetidos para hacer sprites individuales los cuales se puedan diferenciar en un futuro con otro tipo de sprites para el camino y para el inicio y final del mismo, porque alargar la escala de un sprite, al final quedaría mal, aunque también se podría tener un sprite del pasillo si sabes la longitud exacta que deseas, igual que las habitaciones están pre-hechas en este caso.

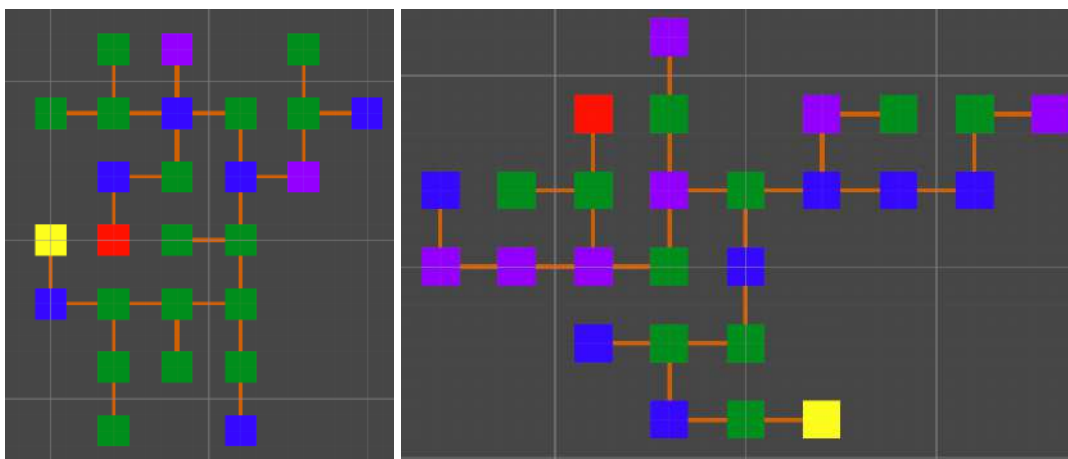


Fig. 39 Generación de 30 habitaciones unidas con los pasillos. Fuente: Propia a través de Unity

Para una versión final, he puesto unos sprites de habitaciones para que no sean sólo colores y ya está, y se pueda ver más inmersivamente cómo sería la generación final de la mazmorra con las habitaciones y los pasillos que el usuario decida.

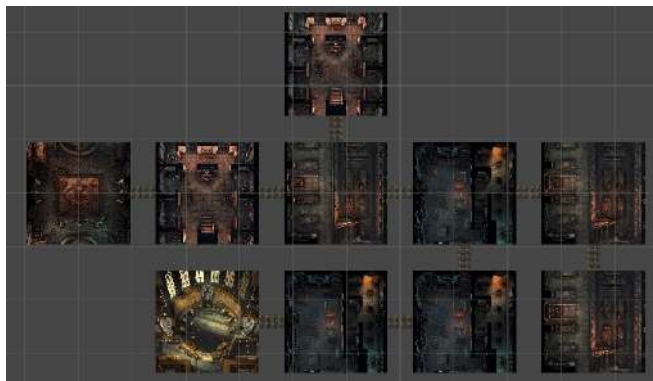


Fig. 40 Generación de 10 salas con los nuevos sprites de habitaciones pre hechas. Fuente: Propia a través de Unity

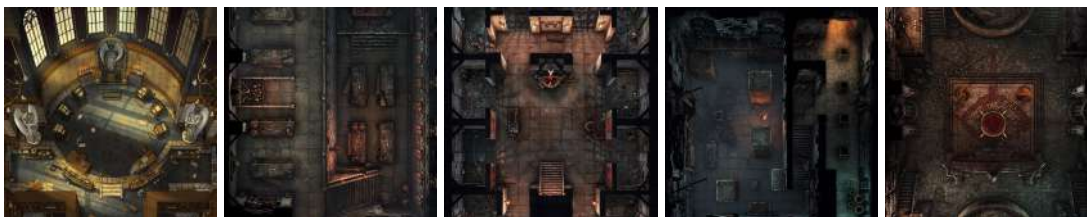


Fig. 41 Los distintos tipos de habitaciones, la inicial, fácil, normal, difícil y boss, respectivamente de izquierda a derecha. Fuente: Fotos recortadas cogidas de pinterest

5.2. Implementación Algoritmo de Partición Binaria del Espacio (BSP)

Para la creación de este algoritmo he mirado una serie de videos de youtube [47] en los cuales el creador implementa en cierto punto una división del espacio de este estilo, por lo que he seguido ciertos de sus pasos de este creador de contenido que lleva años haciendo videos de programación y sobre todo en sus inicios de generación procedural.

También he cogido los propios sprites para ahorrar tiempo.

Lo primero que hay que hacer para esta implementación es tener un "Grid", ya que se va a hacer todo con "tiles", donde tendré un "tilemap" con el suelo, y otro con los distintos tipos de muros.

En esos tiles es donde en el futuro se pueden poner los colliders en el apartado de los muros y así tener la habitación cerrada correctamente.

Una vez con el Grid y con los sprites pertinentes, he implementado la parte de que se divida el espacio y ponga el suelo en un espacio de 100x100 y donde la altura o la anchura, sea de 20 tiles, llamémoslos píxeles, aunque no sea propiamente eso.

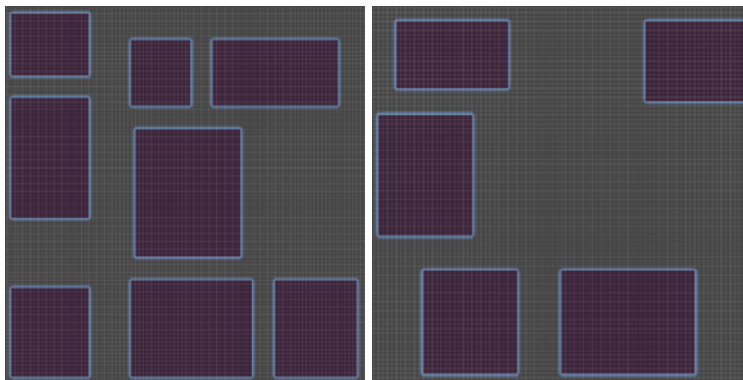


Fig. 42 Generación del suelo con el algoritmo BSP. Fuente: Propia a través de Unity

A partir de aquí, se implementan los muros, para la implementación de los muros hay que hacer muchas comprobaciones, para saber si es el muro derecho, izquierdo, arriba, abajo, o cualquiera de las 4 esquinas.

Esas comprobaciones se hacen con comprobaciones de bits, por ejemplo, para el muro de la posición izquierda, sería 0b0100, los ceros son las posiciones que son muro, o que no hay nada, el espacio vacío, y el uno son las posiciones del suelo, siendo este de cuatro posiciones, el primer bit es el de arriba, el segundo el de la derecha, tercero, abajo, y el último la izquierda.

Por otro lado, para las esquinas, es más complicado, hay que hacerlo con 8 bits, para la esquina superior derecha por ejemplo, serían 0b00000100 y 0b00000101, únicamente por si ocurriese el caso en el que justo en la esquina superior izquierda de donde está puesto este bloque, justo hay otro bloque que hiciese de suelo para otra sala contigua, pero a la que esta no afectase, por eso es necesario poner todas las combinaciones posibles para que no de errores.

Y en estos casos hay pocas opciones, para el muro izquierdo sólo 1 y para la esquina superior exterior derecha sólo dos, pero hay algunos que llegan a tener más de 10 opciones para tener todas las opciones previstas y que no pueda dar error al generar el mapa.

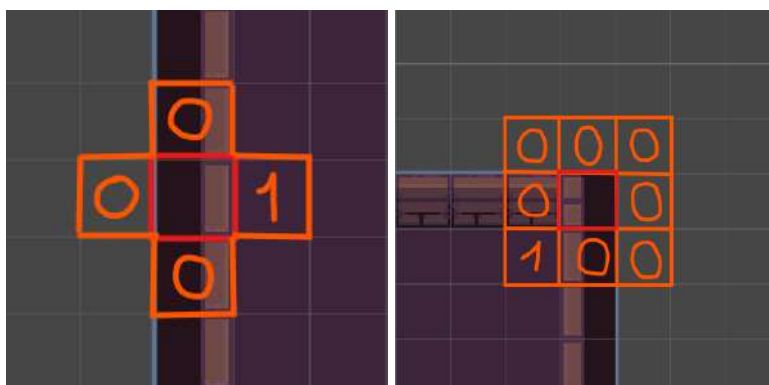


Fig. 43 Cómo se calculan los bits para especificar qué sprite poner en cada parte del mapa. Fuente: Propia a través de Unity

Con todos estos ajustes, y siempre hay que estar revisando por si acaso se ha pasado algún que otro bit que debería ser cero en vez de 1 para poner una u otra opción de sprite, ya tendríamos las salas creadas completamente, como se puede apreciar en la Fig. 42.

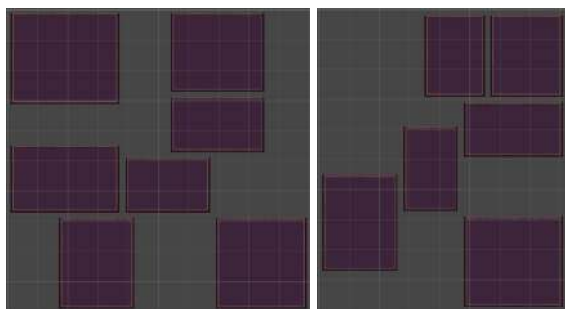


Fig. 44 Generación de las habitaciones con sus muros. Fuente: Propia a través de Unity

Ahora ya sólo falta unir las habitaciones con los pasillos para tener conectada la mazmorra o el mapa al completo y poder tener un mapa jugable.

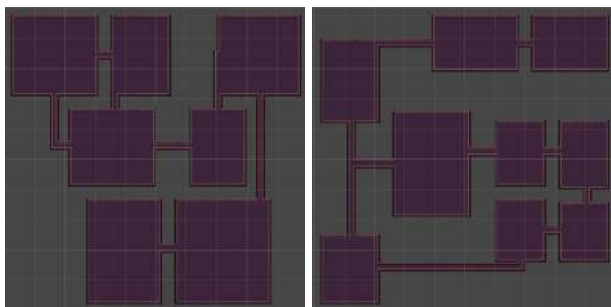


Fig. 45 Generación de las habitaciones conectadas con los pasillos. Fuente: Propia a través de Unity

El problema aquí es que no puedes tener un control exacto de las habitaciones máximas o mínimas que puedes tener, ya que te divide el espacio con el algoritmo y te va buscando huecos para crear las habitaciones según los parámetros que le pongas de máximo y mínimo. Por lo que he hecho unos ajustes para poder tener un número exacto de habitaciones.

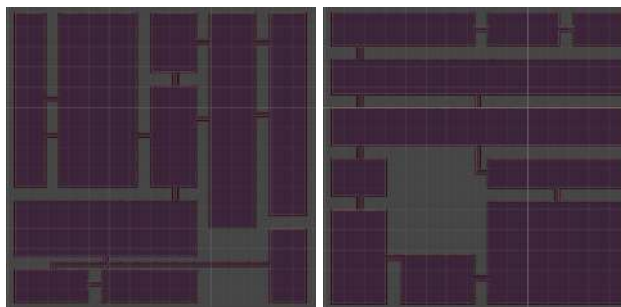


Fig. 46 Generación con un máximo de 10 habitaciones en un espacio de 150x150 y siendo el mínimo de altura y anchura de 20. Fuente: Propia a través de Unity

Así ocupan todo el espacio disponible siempre que tengan espacio para las habitaciones con el mínimo de altura y anchura propuestos, en este caso he tenido que subir del 100x100 a un 150x150 porque si no, no se generaban las 10 habitaciones que había puesto de máximo.

Y otro pequeño ajuste para poder poner el máximo de anchura y altura para que no se creasen esas habitaciones tan largas o enormes, lo cual se puede querer en el juego.

Al final así hay más aleatoriedad ya que ocupan todo el hueco posible creando esas habitaciones grandes, donde en otras ocasiones habría un vacío más grande aún del que se puede apreciar en la imagen de la derecha de la Fig. 44.

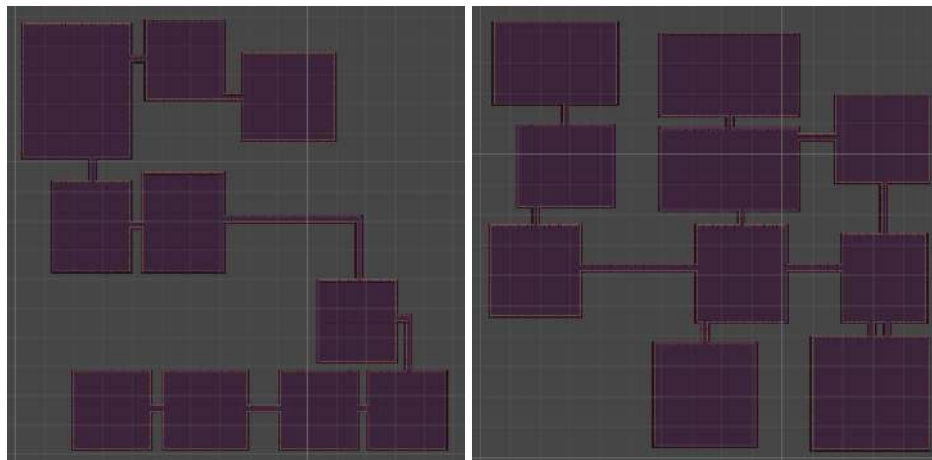


Fig. 47 Generación con un máximo de 10 habitaciones en un espacio de 150x150 y siendo el mínimo y máximo de altura y anchura de 30. Fuente: Propia a través de Unity

El problema de esto es que habría que ir ajustando a mano el tamaño para que las habitaciones se pusieran cerca una de la otra, porque si no, ocupa todo el espacio que le pones de mínimo y máximo y las habitaciones quedan muy separadas unas de las otras como se puede apreciar en la Fig. 45.

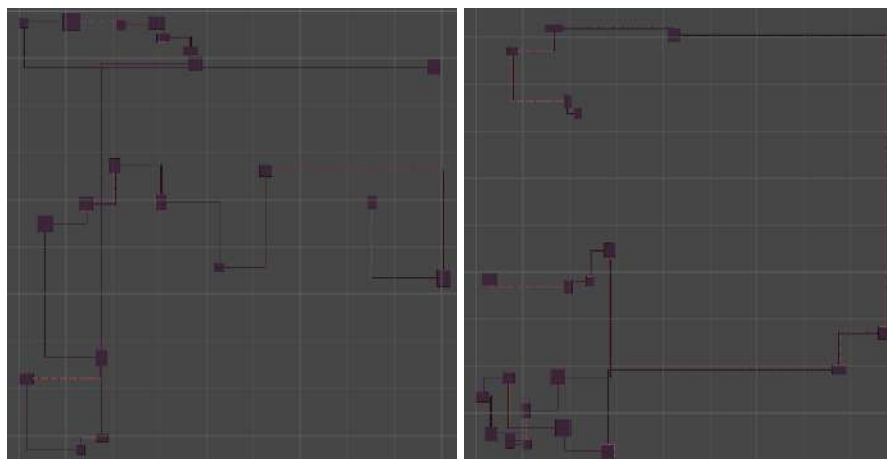


Fig. 48 Generación con un máximo de 20 habitaciones en un espacio de 1000x1000 y siendo el mínimo de anchura y altura de 20 y el máximo de 40. Fuente: Propia a través de Unity

Por lo que para solucionar ese problema he hecho que se calcule automáticamente el espaciado de altura y anchura para que se empiecen a dividir y se creen las habitaciones.

Este espaciado lo he calculado de la siguiente manera:

```
int spacing = offset + 1;
int estimatedArea = maxRooms * (maxRoomWidth + spacing * 2) * (maxRoomHeight + spacing * 2);
int side = Mathf.CeilToInt(Mathf.Sqrt(estimatedArea)) + (spacing * 4);
dungeonWidth = Mathf.Max(side, (minRoomWidth + spacing * 2) * 3);
dungeonHeight = Mathf.Max(side, (minRoomHeight + spacing * 2) * 3);
```

Para que así haya espacio de sobra para que puedan caber todas las habitaciones, y, por si acaso, un bucle con algunas iteraciones por si a la primera no crea el total de habitaciones requeridas.

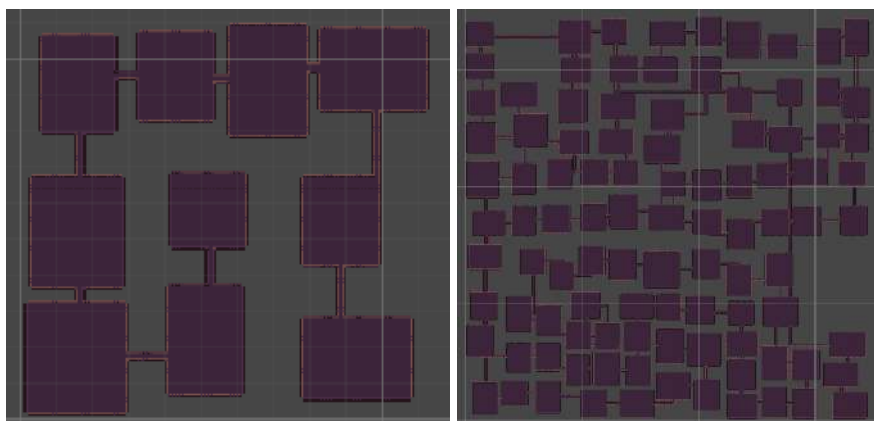


Fig. 49 Generación con un máximo de 10 y 100 habitaciones respectivamente de izquierda a derecha y el mínimo de anchura y altura de 20 y máximo de 30. Fuente: Propia a través de Unity

Con 10 habitaciones y los parámetros anteriores de anchura y altura que están en la foto, el tiempo de ejecución es de 0,388 milisegundos de media, el tiempo de CPU de 0,465 milisegundos y 3 KB.

Con 100 habitaciones y los mismos parámetros da 0,46 milisegundos de tiempo de ejecución, 0,498 milisegundos de CPU y 6 KB.

Para que no esté tan vacío el mapa, he generado aleatoriamente decoraciones a lo largo de todas las habitaciones, con un slider en el que decides si quieres más o menos y respetando las zonas de las entradas y salidas de las habitaciones.

Luego un sistema para definir qué tipo de habitación que es, la inicial, la del boss, las distintas dificultades y alguna sala del tesoro.

Para así a partir de ahí diferenciar lo que puede salir o no en cada tipo de habitación para que haya cosas diferentes en cada una.

Y otro sistema para elegir cuántas habitaciones de cada tipo quieres para tenerlo más específico aún, y que se autocalcule el espacio también como la generación anterior.

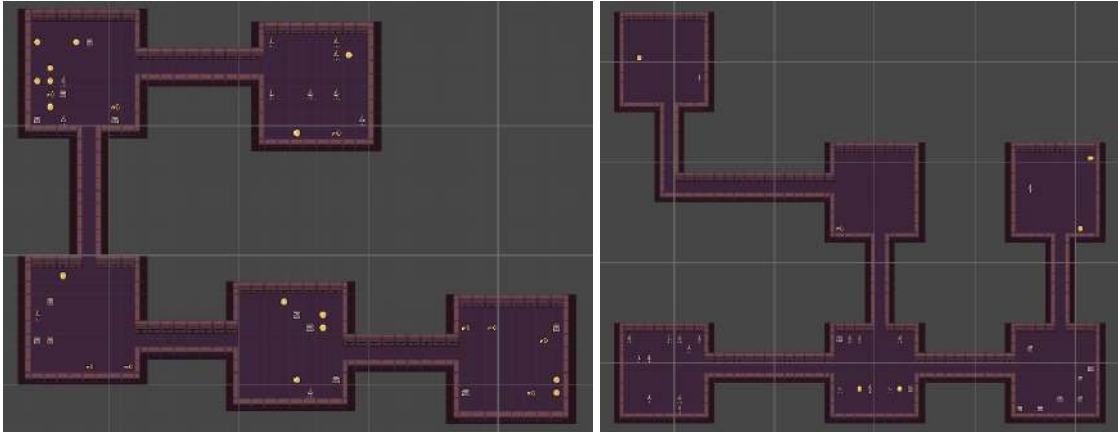


Fig. 50 Generación de la decoración de las salas, a la izquierda la generación aleatoria de todas las decoraciones con el slider, a la derecha la diferenciación de salas con sus decoraciones propias. Fuente: Propia a través de Unity



6. Estudio Económico

Por un lado tendríamos los costes directos propios, donde habría que ver el tiempo invertido a lo largo del tfg, tanto la investigación como la implementación de ambos algoritmos. Ese tiempo se podría estimar como unas 300 horas.

Para saber los costes directos, he cogido el salario bruto anual de un trabajador junior, el cual ronda los 24.000 €, por lo que, contando que son 52 semanas por año, 5 días a la semana y 8 horas al día, eso da 2080 horas al año, así que $24.000 \text{ €} / 2080 \text{ horas} = \sim 11,54 \text{ €} / \text{hora}$.

Multiplicándolo por la aproximación de las 300 horas: $300 \text{ horas} * 11,54 \text{ €} / \text{hora} = 3.462 \text{ €}$.

Por otro lado están los costes indirectos, como serían las licencias de software, el consumo eléctrico... Al cual se le aplica un 30% del coste personal.

Por lo que el total de los costes indirectos sería: $30\% \text{ de } 3.462 \text{ €} = 1.038,60 \text{ €}$.

La inversión total daría $3.462 \text{ €} + 1.038,60 \text{ €} = 4.500,60 \text{ €}$. 4.500 € para redondear.

Este proyecto generaría un valor para la empresa no como un ingreso directo, sino que además es un código reusable en un futuro para otros juegos.

El ahorro en horas de investigación e implementación gracias al trabajo ya realizado, se podría estimar en unas 200 horas de trabajo. Así que las 200 horas, multiplicadas por los euros a la hora calculados previamente, daría: $200 \text{ h} * 11,54 \text{ €/h} = 2.308 \text{ €}$

Gracias al estudio comparativo de los diferentes algoritmos, se pueden evitar fases de optimización posteriores, lo cual podría equivaler a un ahorro de $\sim 1.200 \text{ €}$.

Y como se ha explicado, es un código reusable para futuros proyectos, que le puede servir a la empresa para evitar desarrollar desde 0 de nuevo los diferentes algoritmos, lo cual podría suponer 2.000 € más de ahorro gracias a este activo.

Así pues, el beneficio total para la empresa sería $2.308 \text{ €} + 1.200 \text{ €} + 2.000 \text{ €} = 5.508 \text{ €}$.

La recuperación de la inversión se calcula dividiendo el beneficio neto entre la inversión, multiplicándola por 100. El beneficio neto sería de $5.508 \text{ €} - 4.500 \text{ €} = 1.008 \text{ €}$.

Y el ROI (Retorno de la inversión) de $(1.008 \text{ €} / 4.500 \text{ €}) * 100 = 22.4\%$.

Es decir, por cada euro invertido, la empresa obtendría un retorno de 1,22 euros, es un ROI positivo y superior al 20%, lo cual indica que es una inversión muy rentable.

Análisis de riesgos: El riesgo técnico de que la generación no funcione, o no cumpla las expectativas, está mitigado, gracias al trabajo y comparaciones de los distintos métodos probados en este trabajo, por otro lado, el riesgo en partes del desarrollo del juego también se ven minimizados, ya que la fase de generación de niveles está prácticamente completada.

Para tenerlo todo de una manera más visual, he hecho unas tablas que pueden servir para comprenderlo mejor.

Costes directos:

Concepto	Cálculo	Total
Salario bruto anual		24.000 €
Jornada anual	52 semanas * 5 días * 8 horas	2080 horas
Euros / hora	24.000 € / 2080 horas	11,54 € / hora
Horas dedicadas		300 horas
Coste directo total	300 horas * 11,54 € / hora	3.462 €

Costes indirectos:

Concepto	Cálculo	Total
Porcentaje sobre el coste personal	30%	
Coste indirecto total	30% de 3.462 €	1.038,60 €

Inversión total:

Concepto	Cálculo	Total
Coste directo		3.462,00 €
Coste indirecto		1.038,60 €
Inversión total (redondeado)	3.462,00 € + 1.038,60 €	4.500,00 €

Beneficios y ahorros generados:

Concepto	Cálculo	Valor
Ahorro de horas de desarrollo	200 horas * 11,54 € / hora	2.308 €
Ahorro en optimizaciones	Estimación	1.200 €
Valor del activo reusable	Estimación	2.000 €
Beneficio total	2.308 € + 1.200 € + 2.000 €	5.508 €



Retorno de la Inversión (ROI):

Concepto	Cálculo	Valor
Inversión total		4.500 €
Beneficio total		5.508 €
Beneficio neto	$5.508 \text{ €} - 4.500 \text{ €}$	1.008 €
ROI	$(1.008 \text{ €} / 4.500 \text{ €}) * 100$	22.4%



7. Resultados y evaluación

Al terminar el TFG se han obtenido los siguientes productos:

- Dos sistemas de generación procedural de mapas 2D en el entorno de Unity, uno de ellos es un sistema de grafos, y otro un sistema basado en la partición binaria del espacio.
- Código modular y reusable, adaptable para futuros trabajos o proyectos.
- Variantes de mapas en ambos algoritmos, donde se pueden configurar tamaño, número de salas, distintos métodos para su creación, variedad de decoraciones y capacidad de modificarlo a gusto de la persona.
- Un sistema de métricas de rendimiento recopiladas para cada algoritmo y diferentes casos comunes de número de salas del género.

Evaluación de los resultados:

- Pruebas de rendimiento: Se ha medido el tiempo de ejecución, de CPU y uso de memoria para configuraciones de 10 a 100 habitaciones, con los cuales se ha podido ver lo siguiente, en el caso del algoritmo de grafos:

Con 10 habitaciones:

Método	Tiempo Ejecución (ms)	Tiempo CPU (ms)	Memoria (KB)
Prim Normal	0,057	0,068	0
Prim Mejorado	0,067	0,076	0
Kruskal	0,050	0,060	0
Grid Cuadrado	0,038	0,047	0

En este caso no pongo los distintos tipos de algoritmos con el Grid cuadrado final, ya que los tiempos no varían apenas, donde puede haber más diferencia es en el caso de 100 habitaciones, en el cual sí que voy a diferenciar cada método.

Con 100 habitaciones:

Método	Tiempo Ejecución (ms)	Tiempo CPU (ms)	Memoria (KB)
Prim Normal	3,800	4,500	8,0
Prim Mejorado	1,250	1,800	21,6
Kruskal	1,480	1,770	5,6
Grid Cuadrado - Prim	3,630	4,360	5,6
Grid Cuadrado - Prim Mejorado	0,780	0,940	24,0
Grid Cuadrado - Kruskal	1,400	1,680	7,2

Y luego está el caso del algoritmo de la partición binaria del espacio:

Cantidad Habitaciones	Tiempo Ejecución (ms)	Tiempo CPU (ms)	Memoria (KB)
10 Habitaciones	0,388	0,465	3
100 Habitaciones	0,460	0,498	6

- Pruebas de jugabilidad: Se han ido mejorando los algoritmos para que en los mapas generados todas las salas estén conectadas, sean accesibles y opciones por si se desea que la sala final del boss aparezca más alejada de la sala inicial, como suele ocurrir en los juegos de este estilo. Así como una diferenciación de salas (fáciles, normales, difíciles, del tesoro) por si se desea añadir algún otro tipo de sala especial, y junto a ello la capacidad de poder cambiar la decoración de las distintas salas.
- Errores detectados:
 - En un principio los primeros métodos de generación del algoritmo de grafos, no tenían las habitaciones en su eje vertical y horizontal exactos, lo que dificultaba la generación de los pasillos y más siendo como suelen ser los mapas de este estilo de juegos de mazmorras.
 - La sala del boss podía aparecer junto a la inicial, por lo que implementé una función para que estuviesen más separadas.



- En el algoritmo de BSP, las habitaciones podían estar muy separadas, dependiendo de la altura y anchura, así que implementé un cálculo automático del espacio para que se hagan las habitaciones precisas en un espacio más reducido.

Respecto a cambios en la metodología o los objetivos, no han habido problemas en ese sentido, ya que desde un principio se buscaba crear mapas únicos e investigar diferentes técnicas y tener algún tipo de demostración y ver los pros y contras, y todo eso se ha cumplido. Así como la planificación del mismo.

Respecto a algún riesgo mitigado, por tiempo, podría ser que en la generación de mapas del algoritmo de grafos, está hecho con habitaciones prediseñadas, en vez de que se generen aleatoriamente como pasa con el algoritmo BSP, aunque eso también estaba pensado para ver diferentes formas de crear mapas procedurales, pero sí que es cierto que gracias a hacerlo así, el tiempo de programación de ese algoritmo ha sido menor.

Por otra parte, viendo las pruebas de rendimiento, se puede observar que con 10 habitaciones, la situación no varía mucho entre grafos y BSP, sobre todo con el método final del grid cuadrado y el método de la partición, que son casi idénticos, exceptuando que de primeras tiene más peso de memoria el BSP, sin embargo, al llegar a las 100 habitaciones hay un salto muy grande en general, y en grafos el que más se le puede acercar es el del grid cuadrado con el método de prim mejorado, aunque la memoria es 4 veces superior, y los tiempos casi se duplican, por muy pequeños que sean en cada caso.

Viendo estos resultados así sólamete, se podría decir que el algoritmo de BSP es mucho mejor, menor tiempo y en habitaciones más grandes, menor memoria, pero ciertamente depende del resultado que estés buscando.

Con el algoritmo de grafos, lo puedes tener todo mucho mejor conectado, y con las salas prefabricadas, sólo es necesario diferenciar el nivel de dificultad de cada sala, yo lo he hecho en fáciles, normales y difíciles, y sólo un tipo de habitación por dificultad pero se pueden añadir muchas variaciones y salas secretas, que se necesiten destruir paredes para que aparezcan, o salas del tesoro como he añadido en el BSP, o cualquier cosa que se pueda imaginar, como pasa en The Binding of Isaac, y en vez de pasillos igual se quieren poner puertas que te llevan a la habitación de la dirección por donde has pasado.

Y el algoritmo de partición binaria del espacio sobre todo se usaba en juegos antiguos, como Rogue, o Doom, ya que era más eficiente en esa época, pero sirve para tener una estructura de la geometría de las habitaciones de la mazmorra.

Aunque antes se usaba más porque solucionaba un problema muchos problemas, comentados en el apartado de antecedentes, ya que los ordenadores de los años 80/90 no eran ni de lejos lo que estamos acostumbrados a tener.

Hoy en día hay pocos juegos que se hagan sólomente con un algoritmo, en la combinación está la clave, se puede usar la estructura de la geometría del algoritmo de BSP y la conectividad del algoritmo de grafos para tener un resultado final mucho mejor, en este trabajo sólo quería comprobar ciertas diferencias, pero no solo de rendimiento, si no de acabado, entre estos dos algoritmos, y una vez que los he terminado de hacer, creo que me quedaría más, si me tuviese que quedar sólo con uno de los dos, con el algoritmos de grafos, me gusta más, no sé si porque lo veo más visual o porque pienso que tiene más potencial y te da más opciones, aunque de rendimiento sea algo peor, al menos tal y como lo he planteado.

Y creo que a los jugadores habituales de este género les gustaría más también y sería más disfrutable, ya que lo veo más dinámico y con más opciones para el desarrollador de hacer multitud de cosas más variadas y más acabadas en términos de habitaciones más pulidas y, que aunque haya aleatoriedad, no sea tan impreciso que te puedas encontrar ciertas habitaciones que no tengan el estilo visual y de juego que se desea.

8. Conclusiones

Respecto al cumplimiento de los objetivos planteados en un inicio, yo creo que se han cumplido todos ellos, se ha hecho una comparativa tanto de rendimiento como en general de los diferentes aspectos que engloban la creación de los algoritmos de grafos y de BSP.

Lo cual no quita que se puedan hacer ciertas mejoras o ampliaciones al trabajo ya realizado, como por ejemplo unir ambos algoritmos en un sólo mapa cogiendo lo mejor de cada algoritmo, ya explicado en el apartado anterior, para poder tener un sólo algoritmo combinado que sea capaz de crear salas mucho mejores, no se si de rendimiento, pero estructuralmente debería.

Respecto a mejorar ambos algoritmos de manera individual, también se podría haber hecho ambos con salas prefabricadas como hago en el algoritmo de grafos, o salas que se crean a sí mismas como en el algoritmo de BSP, para poder tener una comparativa mucho más fiable, ya que los datos de ahora pueden no ser del todo fiables, ya que están hechos de diferentes formas porque quería enseñar diversos tipos de creación de salas también.



9. Bibliografía

- [1] Y. Zhang, G. Zhang and X. Huang, "A Survey of Procedural Content Generation for Games," 2022 International Conference on Culture-Oriented Science and Technology (CoST), Lanzhou, China, 2022, pp. 186-190, doi: 10.1109/CoST57098.2022.00046.
- [2] Explorador RPG (2020). Beneath Apple Manor: El primer roguelike. <https://exploradorrpg.wordpress.com/juegos/beneath-apple-manor/>
- [3] LaunchBox (s.f.). Beneath Apple Manor [en línea]. Disponible en: <https://gamesdb.launchbox-app.com/games/details/84157-beneath-apple-manor>
- [4] Wikipedia (s.f.). Rogue (video game) [en línea]. Disponible en: [https://en.wikipedia.org/wiki/Rogue_\(video_game\)](https://en.wikipedia.org/wiki/Rogue_(video_game))
- [5] Wikipedia (s.f.). Maze Craze [en línea]. Disponible en: https://en.wikipedia.org/wiki/Maze_Craze
- [6] Wikipedia (s.f.). Rescue on Fractalus [en línea]. Disponible en: https://en.wikipedia.org/wiki/Rescue_on_Fractalus
- [7] Wikipedia (s.f.). River Raid [en línea]. Disponible en: https://en.wikipedia.org/wiki/River_Raid
- [8] FILFRE.NET (2011). Akalabeth [en línea]. Disponible en: <https://www.filfre.net/2011/12/akalabeth/>
- [9] Wikipedia (s.f.). Elite (videojuego) [en línea]. Disponible en: [https://es.wikipedia.org/wiki/Elite_\(videojuego\)](https://es.wikipedia.org/wiki/Elite_(videojuego))
- [10] Wikipedia (s.f.). Generación por procedimientos [en línea]. Disponible en: https://es.wikipedia.org/wiki/Generaci%C3%B3n_por_procedimientos
- [11] Unity Technologies (s.f.). SpeedTree [en línea]. Disponible en: <https://unity.com/es/products/speedtree>
- [12] Wikipedia (s.f.). .kkrieger [en línea]. Disponible en: <https://en.wikipedia.org/wiki/.kkrieger>
- [13] Wikipedia (s.f.). RoboBlitz [en línea]. Disponible en: <https://es.wikipedia.org/wiki/RoboBlitz>
- [14] Wikipedia (s.f.). Dwarf Fortress [en línea]. Disponible en: https://en.wikipedia.org/wiki/Dwarf_Fortress
- [15] Wikipedia (s.f.). Left 4 Dead [en línea]. Disponible en: https://es.wikipedia.org/wiki/Left_4_Dead
- [16] Wikipedia (s.f.). Left 4 Dead 2 [en línea]. Disponible en: https://es.wikipedia.org/wiki/Left_4_Dead_2
- [17] Wikipedia (s.f.). Development of Spore [en línea]. Disponible en: https://en.wikipedia.org/wiki/Development_of_Spore
- [18] Blind (s.f.). [Video] [en línea]. YouTube. Disponible en: <https://www.youtube.com/watch?v=gorwik2kHyE>
- [19] Wikipedia (s.f.). Minecraft [en línea]. Disponible en: <https://es.wikipedia.org/wiki/Minecraft>
- [20] Dawnosaur (s.f.). How Minecraft Generates Worlds [en línea]. Substack. Disponible en: <https://dawnosaur.substack.com/p/how-minecraft-generates-worlds-you>
- [21] Wikipedia (s.f.). The Binding of Isaac [en línea]. Disponible en: https://es.wikipedia.org/wiki/The_Binding_of_Isaac
- [22] Wikipedia (s.f.). Elite: Dangerous [en línea]. Disponible en: https://es.wikipedia.org/wiki/Elite:_Dangerous
- [23] Khatchadourian, R. (2015). World Without End [en línea]. The New Yorker. Disponible en: <https://www.newyorker.com/magazine/2015/05/18/world-without-end-raffi-khatchadourian>
- [24] Wikipedia (s.f.). No Man's Sky [en línea]. Disponible en: https://es.wikipedia.org/wiki/No_Man%27s_Sky

-
- [25] Fast Company (s.f.). How 4 Designers Built a Game With 184 Quintillion Unique Planets [en línea]. Disponible en: <https://www.fastcompany.com/3048667/how-4-designers-built-a-game-with-184-quintillion-unique-planets>
 - [26] BaityBait (s.f.). [Video] [en línea]. YouTube. Disponible en: <https://www.youtube.com/watch?v=dSJ6RBghpLo>
 - [27] Nivel Oculto (s.f.). Moon Hunters: Forja tu leyenda [en línea]. Disponible en: <https://niveloculto.com/moon-hunters-forja-tu-leyenda/>
 - [28] Wikipedia (s.f.). Starbound [en línea]. Disponible en: <https://es.wikipedia.org/wiki/Starbound>
 - [29] Wikipedia (s.f.). Against the Storm [en línea]. Disponible en: https://es.wikipedia.org/wiki/Against_the_Storm
 - [30] Johnson, G. y Togelius, J. (2010). Cellular Automata for Real-Time Generation of Infinite Cave Levels. Disponible en: <http://julian.togelius.com/Johnson2010Cellular.pdf>
 - [31] López, Gabriel Orlando; ROSSI, Matías Javier. Generación procedural de contenido en videojuegos mediante autómatas celulares [en línea]. Universidad Nacional de La Plata, 2022. Disponible en: https://sedici.unlp.edu.ar/bitstream/handle/10915/153552/Documento_completo.pdf?sequence=1&isAllowed=y
 - [32] WhiteBoxDev (s.f.). Procedural Cave Generation Using Cellular Automata [Video]. YouTube. Disponible en: <https://www.youtube.com/watch?v=Z6m7tFztEvv>
 - [33] Sancho, F. (s.f.). Autómatas Celulares en la Generación Procedural. Universidad de Sevilla. Fuente: https://www.cs.us.es/~fsancho/Blog/posts/Automatas_Celulares.md.html
 - [34] Wikipedia (s.f.). Juego de la vida [en línea]. Disponible en: https://es.wikipedia.org/wiki/Juego_de_la_vida
 - [35] Wikipedia (s.f.). Vecindad de Moore [en línea]. Disponible en: https://es.wikipedia.org/wiki/Vecindad_de_Moore
 - [36] Johnson, G. y Togelius, J. (2010). Cellular Automata for Real-Time Generation of Infinite Cave Levels [en línea]. Disponible en: <http://julian.togelius.com/Johnson2010Cellular.pdf>
 - [37] Selviana, R., Lugata, D. y Alimin, A. (2023). Implementasi Generate Map dan Pemunculan Objek secara Acak pada Game 3D menggunakan Bahasa C# dan Metode Perlin Noise di Unity: Studi Kasus pada Game Development. SPIRIT, 15. Disponible en: <https://doi.org/10.53567/spirit.v15i1.279>
 - [38] Ken Perlin. 1985. An image synthesizer. In Proceedings of the 12th annual conference on Computer graphics and interactive techniques (SIGGRAPH '85). Association for Computing Machinery, New York, NY, USA, 287–296. Disponible en: <https://doi.org/10.1145/325334.325247>
 - [39] Wikipedia (s.f.). Perlin noise [en línea]. Disponible en: https://en.wikipedia.org/wiki/Perlin_noise
 - [40] Wikipedia (s.f.). Algoritmo de Kruskal [en línea]. Disponible en: https://es.wikipedia.org/wiki/Algoritmo_de_Kruskal
 - [41] Wikipedia (s.f.). Algoritmo de Prim [en línea]. Disponible en: https://es.wikipedia.org/wiki/Algoritmo_de_Prim
 - [42] Nepozitek, Ondrej. Edgar DotNet [en línea]. Bachelor thesis. Disponible en: https://github.com/OndrejNepozitek/Edgar-DotNet/blob/text/bachelor_thesis.pdf
 - [43] Nepozitek, Ondrej. Graph-based dungeon generator basics [en línea]. Ondra Nepozitek Blog, 2019. Disponible en: <https://ondra.nepozitek.cz/blog/graph-based-dungeon-generator-basics-1/>
 - [44] Nepozitek, Ondrej. Graph-based dungeon generator implementation [en línea]. Ondra Nepozitek Blog, 2019. Disponible en: <https://ondra.nepozitek.cz/blog/graph-based-dungeon-generator-implementation-2/>
-

- [45] Putra, Pratama Aditya; TARIGAN, Jos Timanta; Zamzami, Elviawaty Muisa. Procedural 2D Dungeon Generation Using Binary Space Partition Algorithm and L-Systems [en línea]. 2023. En: IEEE Conference Publication. Disponible en: <https://ieeexplore.ieee.org/abstract/document/10285811>
- [46] Wikipedia (s.f.). Binary space partitioning [en línea]. Disponible en: https://en.wikipedia.org/wiki/Binary_space_partitioning
- [47] SunnyValleyStudio (s.f.). Unity Procedural Dungeon Generation 2D – Introduction [Video]. YouTube. Disponible en: <https://www.youtube.com/watch?v=-QOCX6SVFsk>



10. Anexos

10.1. Propuesta del TFG

Nombre alumno: Héctor Anguita Domingo

Titulación: Grado de Diseño y Desarrollo de Videojuegos

Curso académico: 2024

TÍTULO DEL PROYECTO

Generación procedural de mapas en un entorno 2D.

DESCRIPCIÓN Y JUSTIFICACIÓN DEL TEMA A TRATAR

La generación procedural de mapas permite ofrecer una experiencia de juego única en cada partida, garantizando que cada mapa sea distinto al anterior. Esto no solo enriquece la jugabilidad, sino que también favorece la retención del jugador, al evitar la repetitividad en títulos de avance por niveles. Al incorporar mapas procedurales, los entornos de juego se vuelven dinámicos y menos predecibles, lo que mantiene el interés del jugador y amplía las posibilidades de exploración en cada nivel.

Este proyecto se desarrollará en un entorno de Unity 2D, programado en C#, dado que Unity es una plataforma sólida y bien documentada para el desarrollo de videojuegos, y es el entorno con el que tengo mayor experiencia.

La elección de este tema se fundamenta en el interés personal por crear videojuegos que se distingan por su unicidad y variedad en los entornos generados. La generación procedural de mapas representa una técnica clave para innovar en el diseño de niveles y optimizar la experiencia del jugador.

OBJETIVOS DEL PROYECTO

Crear un entorno de juego en Unity 2D que utilice generación procedural para diseñar mapas únicos y funcionales en cada partida.

Investigar e implementar distintas técnicas de generación procedural, evaluando la eficiencia de cada una y sus implicancias en la jugabilidad y rendimiento del entorno.

Registrar detalladamente el proceso de diseño, los retos técnicos enfrentados y los resultados producidos.

DOCENTE QUE RESPALDA LA PROPUESTA

Sergio Jimeno



10.2. Acta de reuniones

REUNIÓN: 01

Fecha: 26/11/2024

Lugar: Presencial

Elabora acta: Héctor Anguita

Convocados: Héctor Anguita, Sergio Jimeno

Orden del día / Acta

No.	Asunto	Acuerdo
1	Revisar la propuesta para empezar el trabajo	X
2	Comenzar a mirar artículos y buscar información sobre el tema del trabajo	001

Resumen de acuerdos

Número	Acuerdo	Plazo	Responsable
001	Comenzar a mirar artículos y buscar información sobre el tema del trabajo	Próxima reunión	Héctor Anguita

REUNIÓN: 02

Fecha: 16/06/2025

Lugar: Remoto usando Discord

Elabora acta: Héctor Anguita

Convocados: Héctor Anguita, Sergio Jimeno

Orden del día / Acta

No.	Asunto	Acuerdo
1	Revisar el trabajo realizado	X
2	Ayudar en la estructura para la entrega intermedia del TFG	X
3	Plantear cómo realizar la parte de programación	X
4	Redactar la mayoría de los apartados para la entrega intermedia del TFG	001

Resumen de acuerdos

Número	Acuerdo	Plazo	Responsable
001	Redactar la mayoría de los apartados para la entrega intermedia del TFG	Próxima reunión	Héctor Anguita

REUNIÓN: 03

Fecha: 20/06/2025

Lugar: Remoto usando Discord

Elabora acta: Héctor Anguita

Convocados: Héctor Anguita, Sergio Jimeno

Orden del día / Acta

No.	Asunto	Acuerdo
1	Revisar los apartados realizados	X
2	Tener todo listo para la entrega intermedia del TFG	X

Resumen de acuerdos

Número	Acuerdo	Plazo	Responsable
001	Tener todo listo para la entrega intermedia del TFG	Próxima reunión	Héctor Anguita

REUNIÓN: 04

Fecha: 1/07/2025

Lugar: Remoto usando Discord

Elabora acta: Héctor Anguita

Convocados: Héctor Anguita, Sergio Jimeno

Orden del día / Acta

No.	Asunto	Acuerdo
1	Revisar la entrega intermedia del TFG para poder subirla	X
2	Avanzar en la redacción de los apartados y seguir programando	001

Resumen de acuerdos

Número	Acuerdo	Plazo	Responsable
001	Avanzar en la redacción de los apartados y seguir programando	Próxima reunión	Héctor Anguita

REUNIÓN: 05

Fecha: 2/09/2025

Lugar: Remoto usando Discord

Elabora acta: Héctor Anguita

Convocados: Héctor Anguita, Sergio Jimeno

Orden del día / Acta

No.	Asunto	Acuerdo
1	Revisar el trabajo final	X
2	Mejorar ciertos aspectos visuales	001
3	Terminar de redactar los últimos apartados	002

Resumen de acuerdos

Número	Acuerdo	Plazo	Responsable
001	Mejorar ciertos aspectos visuales	Próxima reunión	Héctor Anguita
002	Terminar de redactar los últimos apartados	Próxima reunión	Héctor Anguita

REUNIÓN: 06

Fecha: 8/09/2025

Lugar: Remoto usando Discord

Elabora acta: Héctor Anguita

Convocados: Héctor Anguita, Sergio Jimeno

Orden del día / Acta

No.	Asunto	Acuerdo
1	Reescribir ciertos apartados	X
2	Revisar y terminar el trabajo	X

Resumen de acuerdos

Número	Acuerdo	Plazo	Responsable
--------	---------	-------	-------------

10.3. Cambios respecto a la entrega intermedia

Los apartados de resumen y de abstract han sido modificados parcialmente debido a que en la entrega intermedia en esos apartados hablaba de que iba a hacer en un futuro, por lo que, una vez finalizado el trabajo, los he reescrito para hablar de lo que he hecho en el trabajo.